*Last updated on **Apr 27, 2026***

Dashboards

You can use dashboards to build data visualizations and share reports with your team. AI/BI dashboards feature AI-assisted authoring, an enhanced visualization library, and a streamlined configuration experience so that you can quickly transform data into sharable insights. When published, your dashboards can be shared with anyone registered to your Databricks account, even if they don't have access to the workspace.

Author dashboards

Author dashboards

Learn how to create, view, organize, and delete dashboards. Collaborate on drafts and use keyboard shortcuts for efficient editing.

Datasets

Define datasets from tables, views, or custom queries to power your dashboard visualizations.

Visualizations

Create charts and visualizations using AI-assisted authoring or manual configuration. Explore visualization types and formatting options.

Filters

Apply global, page-level, and widget-level filters to focus on specific data. Use cross-filtering and drill-through for interactive exploration.

Custom calculations

Create calculated fields and level-of-detail expressions to analyze your data without modifying the dataset.

Dashboard settings

Customize your dashboard's theme, colors, fonts, and locale settings.

Share and collaborate

Publish and share dashboards

Publish dashboards with shared or individual data permissions. Share at workspace and account levels.

Schedule and subscribe

Set up automated refresh schedules and email or Slack subscriptions for published dashboards.

Embed dashboards

Embed published dashboards in external websites and applications using iframes.

Genie Spaces

Enable natural language data exploration by publishing Genie Spaces with your dashboards.

Automate and integrate

Import and export

Transfer dashboards across workspaces by exporting and importing dashboard files. Edit dashboard files directly for advanced customization.

Source control

Use Git integration to version control your dashboards and collaborate with teams.

Declarative Automation Bundles

Manage dashboards programmatically using Declarative Automation Bundles for infrastructure-as-code workflows.

REST APIs

Automate dashboard creation, management, and sharing using Databricks REST APIs.

Lakeflow Jobs

Schedule dashboard updates using Lakeflow Jobs for automated workflow orchestration.

Monitor usage

Monitor usage

Track dashboard creation, views, and interactions using audit logs and system tables.

Next steps

- **Get started with tutorials**: Follow step-by-step guides to create your first dashboard, use AI-assisted visualizations, and learn advanced techniques. See Dashboard tutorials.
- **Explore example dashboards**: Clone and customize pre-built dashboards to jump-start your analytics. See Create a dashboard.
- **Optimize performance**: Learn about caching strategies and optimization techniques for large datasets. See Dataset optimization and caching.
- **Review limits**: Review numerical limits for dashboards, including page limits and other quotas. See Dashboard limits and Resource limits.
- **Connect to your data**: Understand how to define datasets and connect to tables, views, and custom queries. See Create and manage dashboard datasets.

*Last updated on **Apr 27, 2026***

Dashboard concepts

AI/BI dashboards are an interactive data visualization platform that allows you to transform data into actionable insights and share them across your organization. Dashboards feature AI-assisted authoring, an enhanced visualization library, and a streamlined configuration experience to help you quickly build and publish reports. When published, your dashboards can be shared with anyone registered to your Databricks account, even if they don't have access to the workspace.

Key features

AI/BI dashboards provide a comprehensive set of features to support data visualization and collaboration:

Feature	Description
AI-assisted authoring	Use Genie Code to generate visualizations from natural language prompts, reducing the time needed to create complex charts and calculations.
Genie for dashboard insights	Ask questions about your dashboard data using natural language. Published dashboards include a companion Genie Space that lets viewers explore data conversationally without relying on predefined visualizations.
Governed data models	Define datasets from tables, views, metric views, or custom SQL queries. Datasets inherit Unity Catalog permissions and are bundled with dashboards when sharing, importing, or exporting.

Feature	Description
Custom calculations	Create calculated measures and dimensions without modifying the source dataset. Build complex metrics using level-of-detail expressions and window functions.
Interactive filtering	Apply global, page-level, and widget-level filters. Enable cross-filtering and drill-through for interactive data exploration.
Account-level sharing	Publish dashboards to share across workspaces and with users who don't have workspace access. Control permissions at workspace and account levels.
Monitoring	Track dashboard activity using audit logs and system tables. Monitor who views, edits, and interacts with your dashboards for usage analysis and compliance.
Automation and source control	Manage dashboards as code using bundles, automate workflows with REST APIs, and version-control your work with Git. Import and export dashboards across workspaces and environments.

Common use cases

Organizations use AI/BI dashboards to solve a wide variety of business challenges:

Use case	Description
Executive reporting	Create comprehensive executive dashboards that track KPIs, business metrics, and operational performance across departments.
Sales analytics	Monitor sales pipeline, track deal progress, analyze revenue trends, and measure team performance with interactive visualizations.

Use case	Description
Marketing insights	Measure campaign effectiveness, analyze customer engagement, and track conversion metrics to optimize marketing strategies.
Operational monitoring	Track system health, monitor application performance, and visualize operational metrics for DevOps and site reliability teams.
Financial reporting	Build financial dashboards for budget tracking, expense analysis, and forecasting with drill-down capabilities.
Customer analytics	Visualize customer behavior, track retention metrics, and analyze user journeys to improve customer experience.

Dashboard components

AI/BI dashboards consist of two main areas: the **Data** tab where you define datasets and calculations, and the **Canvas** tab where you build visualizations and interactive filters. The canvas can be organized into multiple pages to structure complex reports and minimize scrolling. For details on pages, see Create multi-page reports.

Data modeling

Datasets

Datasets define the data sources that power your dashboard visualizations. Each dataset is based on a table, view, metric view, or custom SQL query.

Dataset type	Description
Table or view	Connect directly to Unity Catalog tables or views. The dataset inherits the schema from the source object.
Metric view	Use metric views to pre-define dimensions, measures, and aggregations. Metric views provide consistent metrics across multiple dashboards.
Custom SQL query	Write custom SQL queries to join tables, filter data, or transform columns before visualization.

Datasets support the following features:

- **Custom calculations**: Define calculated measures and dimensions specific to the dataset
- **Prompt matching**: Enable natural language prompts when using Genie Code
- **Caching**: Improve performance with automatic caching for datasets under 100MB and under 100,000 rows
- **Bundled sharing**: Datasets are included when you share, export, or import dashboards

For details, see Create and manage dashboard datasets.

Custom calculations

Custom calculations let you create new metrics and transformations without modifying your source data. You can define up to 200 custom calculations per dataset.

Calculation type	Description
Calculated measures	Aggregated values such as total revenue, average cost, or profit margin. Calculated measures automatically adjust to the

Calculation type	Description
	dimensions in your visualization.
Calculated dimensions	Unaggregated values or transformations such as categorizing age ranges, concatenating fields, or formatting dates.
Level of detail expressions	Control aggregation granularity independently of your visualization groupings. Use fixed LOD for dataset-wide aggregates or exclude LOD for coarser-grained calculations.
Window functions	Perform calculations across a set of rows, such as rolling averages, cumulative sums, or trailing N-day metrics.

Custom calculations support SQL expressions including aggregate functions, window functions, and conditional logic. You can reference other calculations to build complex metrics incrementally.

For details, see What are custom calculations? and Level of detail (LOD) expressions.

Visualizations

AI/BI dashboards provide a rich library of visualization types to display your data effectively. You can create visualizations manually or use Genie Code to generate them from natural language prompts.

Category	Visualization types
Chart visualizations	Bar, line, area, scatter, pie, combo, funnel, and more
Table visualizations	Pivot tables, detail tables with sorting and formatting
Statistical visualizations	Box plots, histograms, heatmaps for distribution analysis

Category	Visualization types
Map visualizations	Choropleth maps and point maps for geographic data
Specialized visualizations	Gauge charts, single value indicators, and custom formatting

Each visualization can be configured with:

- **Axes and encodings**: Map data fields to x-axis, y-axis, color, size, and other visual properties
- **Aggregations**: Choose aggregation functions like sum, average, count, min, max, and more
- **Sorting and limits**: Control the order and number of data points displayed
- **Formatting**: Customize colors, labels, legends, and tooltips
- **Conditional formatting**: Apply color rules based on data values

For details, see Dashboard visualizations.

Interactivity

Filters and interactive features allow viewers to explore dashboards dynamically and focus on specific data. AI/BI dashboards support multiple filter scopes and types.

Filter scope	Description
Global filters	Apply across all pages and visualizations in the dashboard. Use for high-level filtering like date ranges or regions.
Page-level filters	Apply only to visualizations on the current page. Use to create page-specific views of the data.
Widget-level filters	Apply to individual visualizations. Use for visualization-specific filtering without affecting other widgets.

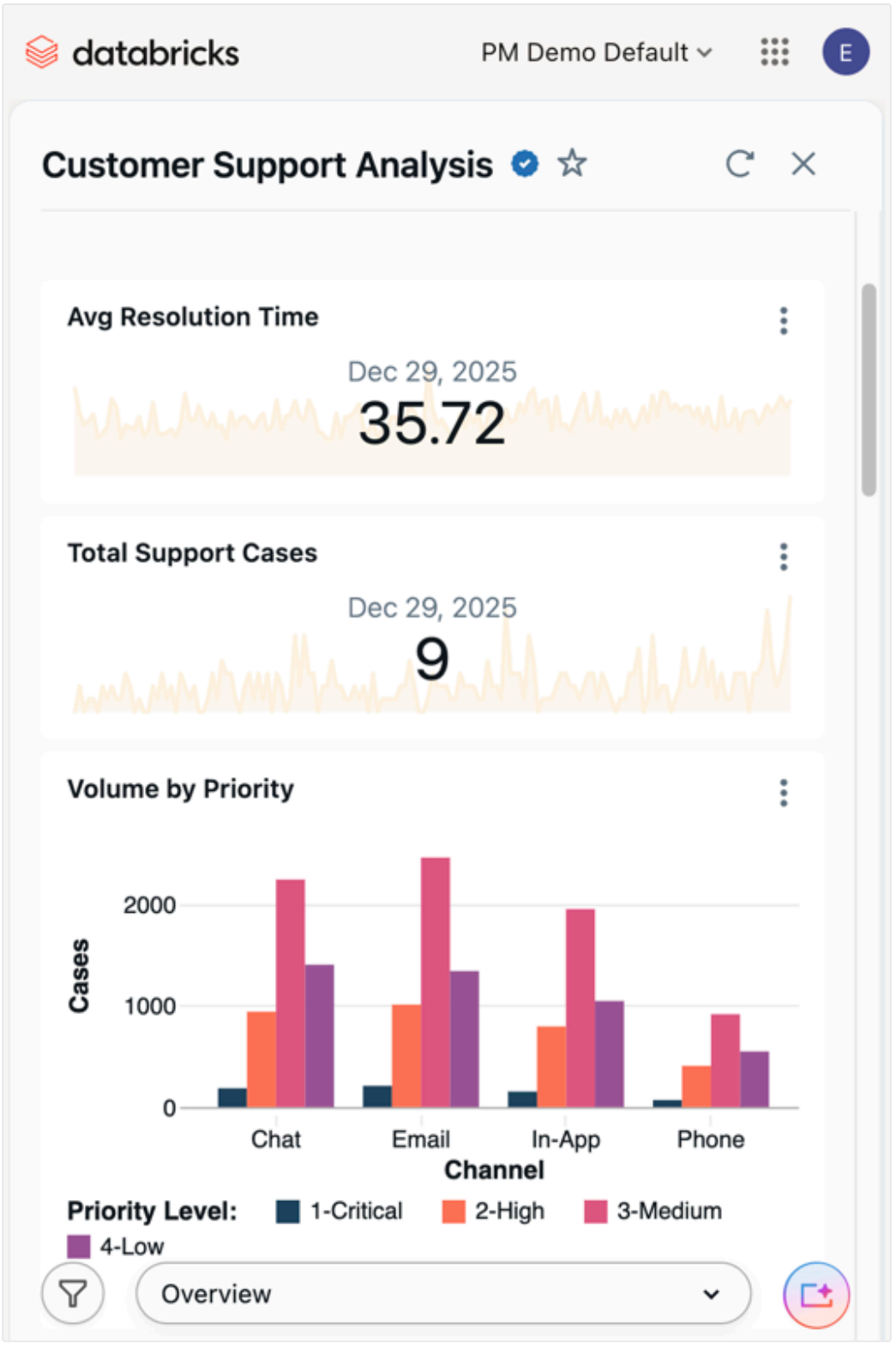
Interactive capabilities include:

- **Field filters**: Filter on specific columns with operators like equals, contains, greater than, and more
- **Parameters**: Create dynamic filters that can be referenced in SQL queries and calculations
- **Cross-filtering**: Click on data points in one visualization to filter others on the same page
- **Drill-through**: Navigate between dashboard pages while preserving filter context

For details, see Use dashboard filters.

Mobile layout

When a dashboard is viewed on a small screen in Genie, it automatically switches to a mobile-friendly layout. No configuration is required. Widgets reflow to a single column and spacing adjusts for touch interaction.



Publishing and sharing

Dashboards support two states: **draft** and **published**. Changes to a draft dashboard are saved automatically but are not visible to viewers until you publish.

Publishing

When you publish a dashboard, you create a snapshot that viewers can access. The published version remains unchanged until you publish again, even if you continue editing the draft.

Publishing options:

- **Shared data permissions**: Viewers see data based on their own Unity Catalog permissions. This ensures data governance and row-level security are enforced.
- **Individual data permissions**: Viewers see data based on the publisher's permissions. This simplifies sharing but requires careful permission management.

Sharing levels

Sharing level	Description
Workspace sharing	Share with users and groups within the same workspace. Users must have workspace access to view the dashboard.
Account-level sharing	Share with users across all workspaces in your account. Users can view the dashboard even without workspace access, enabling broader collaboration.

Additional sharing capabilities:

- **Embedding**: Embed published dashboards in external websites and applications using iframes
- **Schedules and subscriptions**: Set up automated refresh schedules and email or Slack notifications for stakeholders
- **Genie Spaces**: Publish companion Genie Spaces to enable natural language data exploration alongside your dashboards

For details, see Share a dashboard and Manage scheduled dashboard updates and subscriptions.

Draft and published workflows

Understanding the relationship between draft and published dashboards is important for collaboration and maintenance:

- **Draft dashboards**: All edits happen in draft mode. Multiple users with edit permissions can collaborate on a draft. Changes are auto-saved but not visible to viewers.
- **Publishing**: When ready, click **Publish** to create or update the published version. You can optionally include a changelog message.
- **Viewer experience**: Viewers always see the last published version, not the current draft. This prevents work-in-progress changes from affecting stakeholders.
- **Discard changes**: If you want to revert your draft to match the last published version, use **Discard changes** from the dashboard menu.

Performance and optimization

AI/BI dashboards are optimized for performance with several built-in features:

- **Client-side caching**: Datasets under 100MB and 100,000 rows are cached in the browser for faster visualization updates
- **Query optimization**: Dashboards automatically optimize queries to reduce compute usage and improve response times
- **Incremental loading**: Large datasets are loaded progressively to maintain responsiveness
- **Scheduled refresh**: Configure refresh schedules for published dashboards to pre-compute results

For details, see Dataset optimization and caching.

Governance and security

AI/BI dashboards integrate with Databricks security and governance features:

Security feature	Description
Unity Catalog integration	All data access is governed by Unity Catalog permissions, including row filters and column masks
Permission levels	Control who can view, edit, or manage dashboards using workspace and account-level permissions
Audit logs	Track dashboard access and modifications using audit logs and system tables
Tags	Organize and categorize dashboards using tags for governance and discovery

For details, see Monitor dashboard usage.

Automation and integration

AI/BI dashboards support programmatic workflows for teams that need to automate dashboard management:

- **Import and export**: Transfer dashboards across workspaces by exporting and importing JSON files
- **Git integration**: Version control your dashboards using Git repos and sync changes across environments
- **Declarative Automation Bundles**: Manage dashboards as code using Declarative Automation Bundles for infrastructure-as-code workflows
- **REST APIs**: Automate dashboard creation, updates, and sharing using Databricks REST APIs
- **Lakeflow Jobs**: Schedule dashboard updates and refreshes using Lakeflow Jobs

For details, see Dashboard developer workflows.

Next steps

- **Create your first dashboard**: Follow the quickstart tutorial to build a dashboard from scratch. See Create a dashboard.
- **Use AI-assisted visualizations**: Learn how to use Genie Code to create a dashboard using natural language prompts. See Use Genie Code for dashboard authoring.
- **Explore datasets**: Understand how to define datasets from tables, views, and custom queries. See Create and manage dashboard datasets.
- **Build custom calculations**: Create calculated measures and level-of-detail expressions. See What are custom calculations?.
- **Set up filters**: Add interactive filtering to enable data exploration. See Use dashboard filters.

Last updated on ***Apr 9, 2026***

Dashboard tutorials

Follow along with tutorials designed to teach you build and manage AI/BI dashboards.

Get started

If you're new to working with dashboards on Databricks, use the following tutorials to familiarize yourself with some of the available tools and features.

Tutorial	Description
Create a dashboard	Create your first dashboard using a sample dataset.
Use Genie Code for dashboard authoring	Use natural language prompts to generate visualizations on the dashboard canvas.
Tutorial: What If analysis with Genie Code	Use Genie Code for dashboard authoring to build a What If analysis dashboard using avocado sales data.

Apply filters

Filters allow viewers to interact with your dashboards and explore data. You can set up filters to restrict dataset results based on field values or to insert parameter values into a dataset query at runtime.

Tutorial	Description
Use query-based parameters	Set up query-based parameters.

Use Databricks APIs to manage dashboards

Dashboards are data objects that you can manage using Databricks REST APIs. You can use the Workspace API to work with them as generic workspace objects or use Lakeview API tools, which are specifically designed for dashboard management.

The tutorials listed in this section are designed to help you manage dashboards throughout their lifecycle and move dashboards across different workspaces.

Tutorial	Description
Use dashboard APIs to create and manage dashboards	Recommended for migrating dashboards. The Lakeview API is designed to work with dashboard objects. Certain functionality in the Lakeview API mirrors available tools in the Workspace API but includes additional functionality specifically for dashboard management. This tutorial includes examples that demonstrate how to use Lakeview APIs to manage dashboards.
Manage dashboard permissions using the Workspace API	Manage dashboard permissions using the Workspace API.
Manage dashboards with Workspace APIs	If you're already using the Workspace API to manage workspace objects like notebooks, you can continue to use it for many dashboard management operations. This tutorial includes examples that demonstrate how to use the Workspace and Lakeview APIs to manage dashboards.

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

Last updated on ***Feb 25, 2026***

Create a dashboard

Learn how to use the AI/BI dashboard UI to create and share insights. For information about dashboard features, see Dashboards.

The steps in this tutorial demonstrate how to build and share the following dashboard:

You can also create a dashboard using the Dashboard Authoring Agent, see Use Genie Code for dashboard authoring.

Requirements

- You are logged into a Databricks workspace.

Ask Genie

- You have the Databricks SQL entitlement in that workspace.
- You have at least CAN USE access to one or more SQL warehouses.

Step 1. Create a dashboard

Click **New** in the sidebar and select **Dashboard**.

By default, your new dashboard is automatically named with its creation timestamp and stored in your `/Workspace/Users/<username>` directory.

> **NOTE**
>
> You can also create a new dashboard from the Dashboards listing page or the **Create** button in the Workspace menu.

Step 2. Define datasets

The **Canvas** tab is for creating and editing widgets like visualizations, text boxes, and filters. The **Data** tab is for defining the underlying datasets used in your dashboard.

> **NOTE**
>
> All users can write SQL queries to define a dataset. Users in Unity Catalog-enabled workspaces can instead select a Unity Catalog table or view as a dataset.

1. Click the **Data** tab.
2. Click **Create form SQL**.
3. Paste the following query into the editor. Then click **Run** to return a collection of records.

SQL

```
SELECT
 T.tpep_pickup_datetime,
 T.tpep_dropoff_datetime,
```

Ask Genie

```
  T.fare_amount,
  T.pickup_zip,
  T.dropoff_zip,
  T.trip_distance,
  T.weekday,
  CASE
    WHEN T.weekday = 1 THEN 'Sunday'
    WHEN T.weekday = 2 THEN 'Monday'
    WHEN T.weekday = 3 THEN 'Tuesday'
    WHEN T.weekday = 4 THEN 'Wednesday'
    WHEN T.weekday = 5 THEN 'Thursday'
    WHEN T.weekday = 6 THEN 'Friday'
    WHEN T.weekday = 7 THEN 'Saturday'
    ELSE 'N/A'
  END AS day_of_week
FROM
  (
    SELECT
      dayofweek(tpep_pickup_datetime) as weekday,
      *
    FROM
      `samples`.`nyctaxi`.`trips`
    WHERE
      trip_distance > 0
      AND trip_distance < 10
      AND fare_amount > 0
      AND fare_amount < 50
  ) T
ORDER BY
  T.weekday
```

4. Inspect your results. The returned records appear in the **Result table** when the query is finished running.

5. Change the name of your query. Your newly defined dataset is autosaved with the name, **Untitled dataset**. Double click on the title to rename it **Taxicab data**.

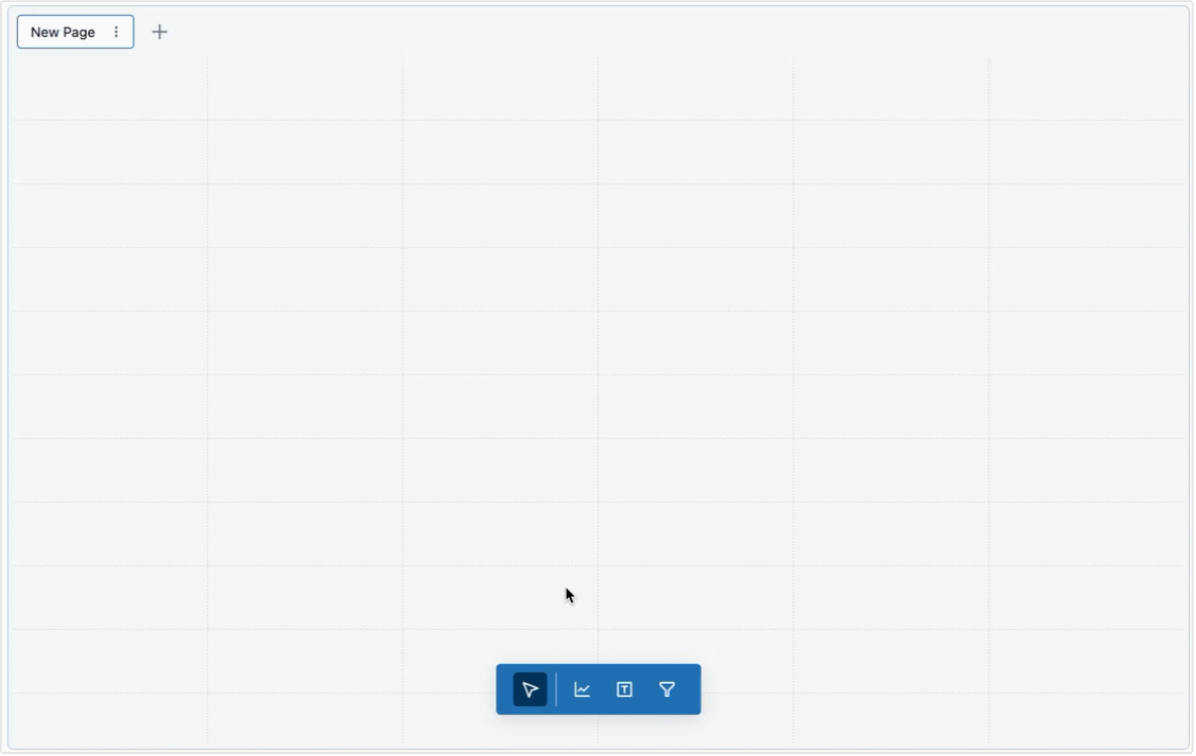
> **NOTE**
>
> This query accesses data from the **samples** catalog on Databricks. The table includes publicly available taxicab data from New York City in 2016. Query results are limited to valid rides that are under 10 miles and cost less than fifty dollars.

Step 3. Add a visualization

Ask Genie

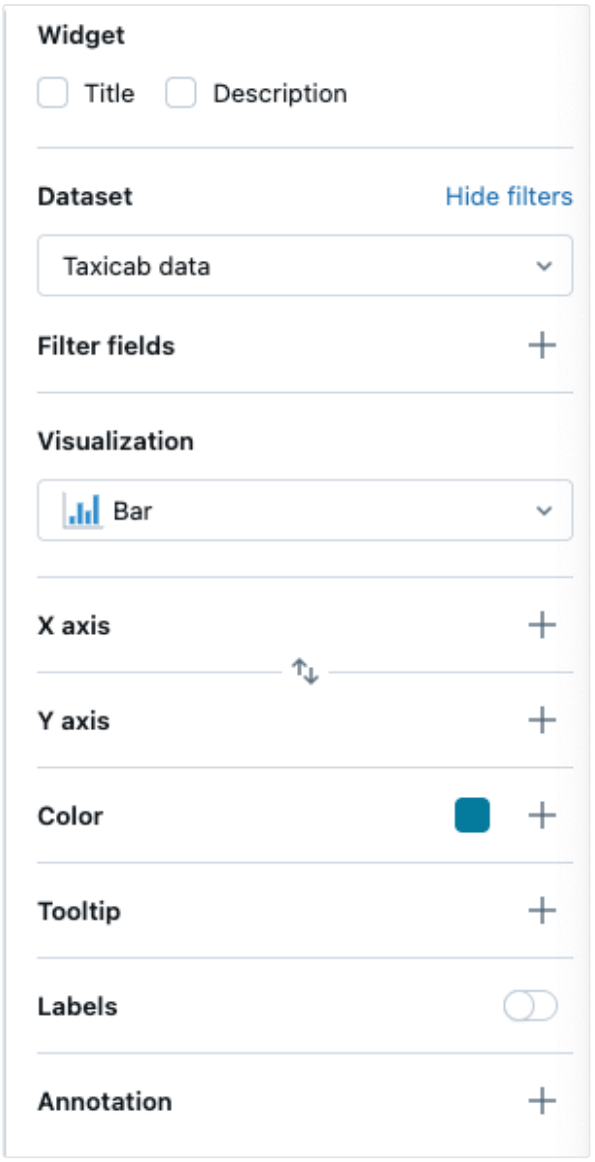
To create your first visualization, complete the following steps:

1. Click the **Canvas** tab.
2. Click **Add a visualization** to add a visualization widget and use your mouse to place it on the canvas.



Step 4. Configure your visualization

When a visualization widget is selected, you can use the configuration panel on the right side of the screen to display your data. As shown in the following image, only one **Dataset** has been defined, and it is selected automatically.



Setup the X-axis

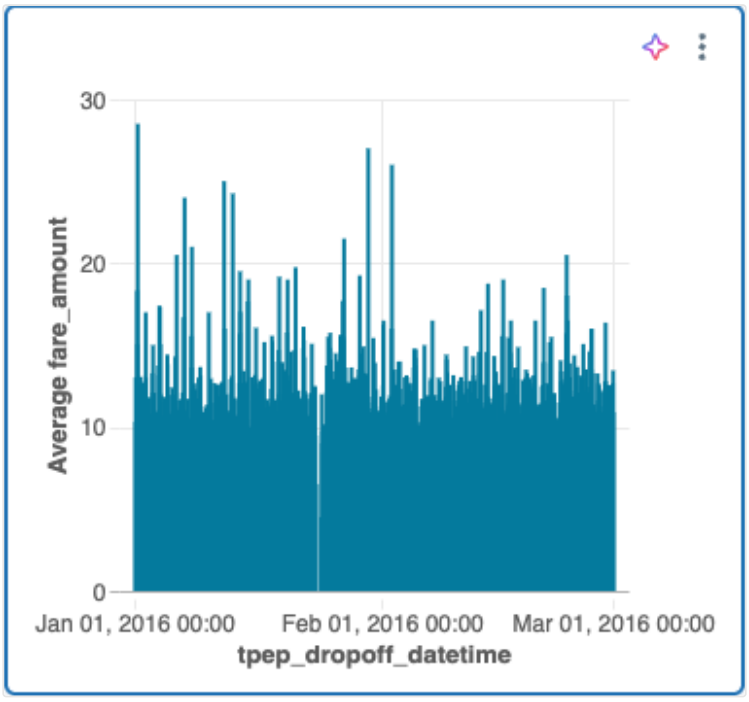
1. If necessary, select **Bar** from the **Visualization** drop-down menu.
2. Click the + plus icon to choose the data presented along the **X-axis**. You can use the search bar to search for a field by name. Select **tpep_dropoff_datetime**.
3. Click the field name you selected to view additional configuration options.
   - As the **Scale Type**, select **Continuous**.
   - For the **Transform** selection, choose **HOURLY**.

Setup the Y-axis

1. Click the + plus icon next to the **Y-axis** to select the **fare_amount** for the data presented along the y-axis.

2. Click the field name you selected to view additional configuration options.
    - As the **Scale Type**, select **Continuous**.
    - For the **Transform** selection, choose **AVG**.

Your chart should look like the follwong example:



Optional: Create visualizations with Genie Code

You can create visualizations using natural language with the Genie Code.

To generate the same chart as above, choose one of the following options:

- To create a new visualization widget:
- From an AI/BI dashboard, open Genie Code side panel.
- In the bottom right corner, select **Agent**. This toggles on Genie Code's Agent mode, allowing you to interact with Genie Code for dashboard authoring.
    - Type “Bar chart of average fare amount over hourly dropoff time”. To create a dashboard using Genie Code, see Use Genie Code for dashboard authoring.

Step 5. Clone and modify a visualization

You can clone an existing chart to create a new visualization.

1. Right-click on your existing chart and then click **Clone**.
2. With your new chart selected, use the configuration panel to change the **X-axis** field to **tpep_pickup_datetime**. If necessary, choose **HOURLY** under the **Transform** type.
3. Use the **Color** selector to choose a new color for your new bar chart.

Step 6. Create a scatterplot

Create a new scatterplot with colors differentiated by value. To create a scatterplot, complete the following steps:

1. Click the **Create a visualization** icon to create a new visualization widget.
2. Configure your chart by making the following selections:
   - **Dataset**: **Taxicab data**
   - **Visualization**: **Scatter**
   - **X axis**: **trip_distance**
   - **Y axis**: **fare_amount**
   - **Color**: Click the + plus icon > **day_of_week**

> **NOTE**
> After colors have been auto-assigned by category, you can change the color associated with a particular value by clicking on the color in the configuration panel.

Step 7. Create dashboard filters

You can use filters to make your dashboards interactive. In this step, you create filters on three fields.

Create a date range filter

1. Click **Add a filter (field/parameter)** to add a filter widget.
2. Click **Widget title** and enter **Date range** to retitle your filter.
3. From the **Filter** drop-down menu in the configuration panel, select **Date range picker**.
4. Click the + plus icon next to the **Fields** menu. Click **tpep_pickup_datetime** from the drop-down menu.

Create a single-select drop-down filter

1. Click **Add a filter (field/parameter)** to add a filter widget.
2. Click **Widget title** and enter **Dropoff zip code** to retitle your filter.
3. From the **Filter** drop-down menu in the configuration panel, select **Dropdown (single-select)**.
4. Select the **Title** checkbox to create a title field on your filter. Click on the placeholder title and type **Dropoff zip code** to retitle your filter.
5. From the **Fields** menu, select **dropoff_zip**.

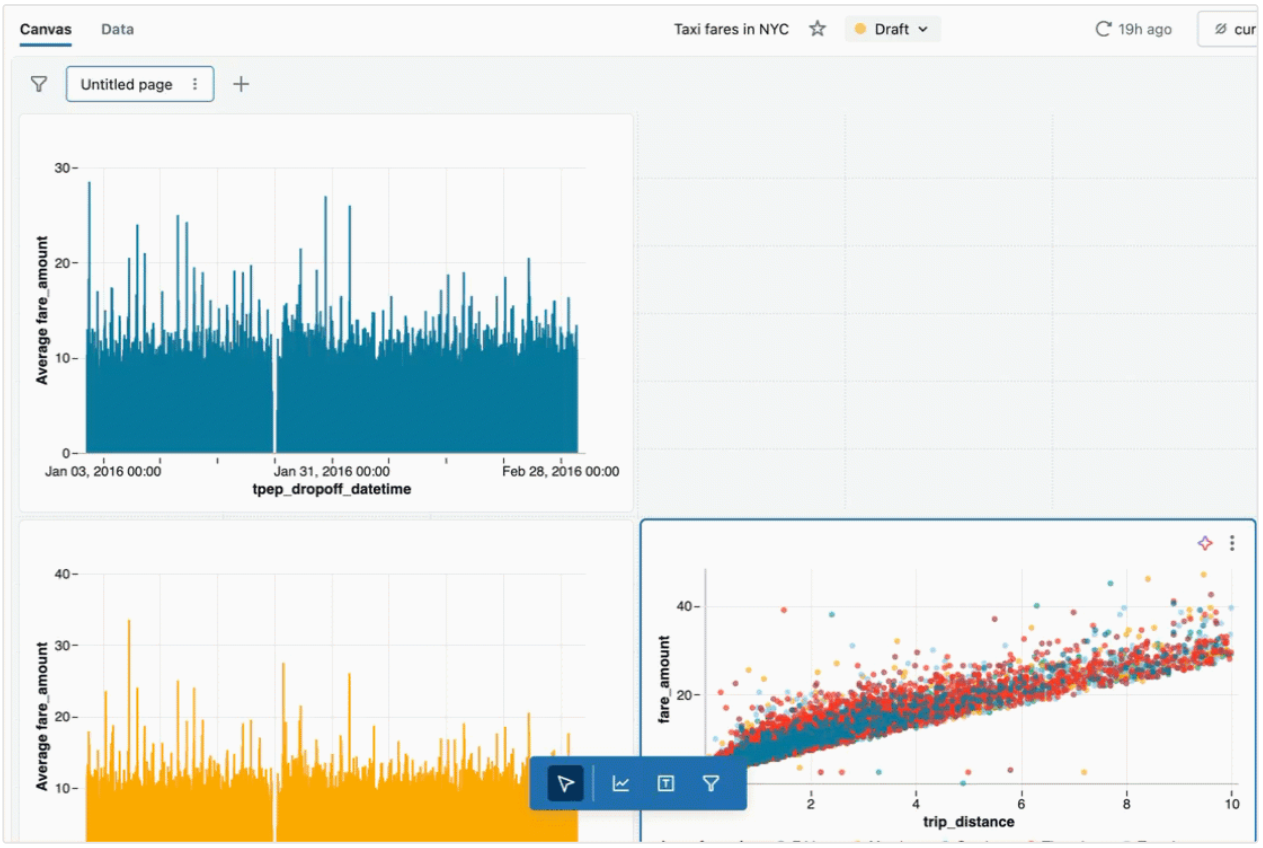
Clone a filter

1. Right-click on your **Dropoff zip code** filter. Then, click **Clone**.
2. Double-click on the title and enter **Pickup zip code** to retitle the cloned widget.
3. Click the — to remove the current field. Then, select **pickup_zip** to filter on that field.

Step 8. Resize and arrange charts and filters

Use your mouse to arrange and resize your charts and filters.

The following image shows one possible arrangement for this dashboard.



Step 9. Publish and share

While you develop a dashboard, your progress is saved as a draft. To create a clean copy for easy consumption, publish your dashboard.

1. Click **Publish** in the upper-right corner of the dashboard.
2. Click **Share with data permissions (default)**.
3. Click **Publish**. A **Sharing** dialog opens.
4. Add users, groups, or service principals that you want to share with. Set permission levels as appropriate. See Share a dashboard and Dashboard ACLs to learn more about permissions and rights.
5. Click **Copy link** and paste it in a new tab to go to your published dashboard.

*Last updated on **Feb 23, 2026***

Use query-based parameters

The article guides you through the steps to create an interactive dashboard that uses query-based parameters. It assumes a basic familiarity with building dashboards on Databricks. See Get started for foundational instruction on creating dashboards.

Requirements

- You are logged into a Databricks workspace.
- You have the Databricks SQL entitlement in that workspace.
- You have at least CAN USE access to one or more SQL warehouses.

Create a dashboard dataset

This tutorial uses generated data from the **samples** catalog on Databricks.

1. Click **New** in the sidebar and select **Dashboard** from the menu.
2. Click the **Data** tab.
3. Click **Create from SQL** and paste the following query into the editor. Then click **Run** to return the results.

```
SQL

SELECT
  *
FROM
  samples.tpch.customer
```

4. Your newly defined dataset is autosaved with the name **Untitled Da** click the title then rename it **Marketing segment**.

Add a parameter

You can add a parameter to this dataset to filter the returned values. The parameter in this example is `:segment`. See Work with dashboard parameters to learn more about parameter syntax.

1. Paste the following `WHERE` clause at the bottom of your query. A text field with the parameter name `segment` appears below your query.

   SQL

   ```
   WHERE
     c_mktsegment = :segment
   ```

2. Type `BUILDING` into the text field below your query to set the default value for the parameter.

3. Rerun the query to inspect the results.

Configure a visualization widget

Add a visualization for your dataset on the canvas by completing the following steps:

1. Click the **Canvas** tab.
2. Click  **Add a visualization** to add a visualization widget and use your mouse to place it in the canvas.

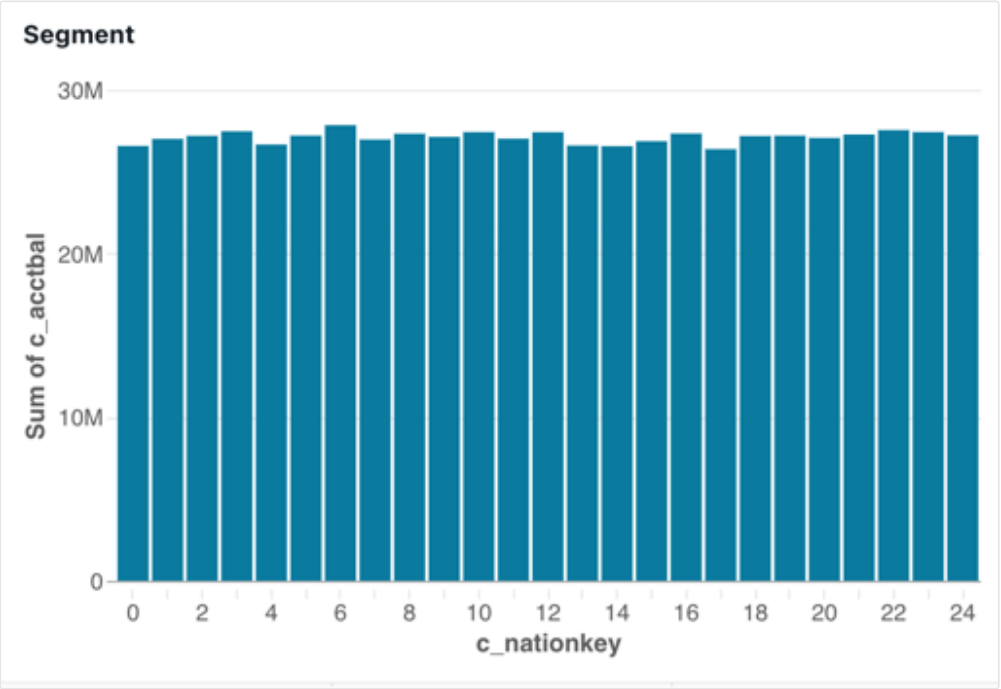
Setup the X-axis

1. If necessary, select **Bar** from the **Visualization** dropdown menu.
2. Click + to choose the data presented along the **X-axis**. You can use the search bar to search for a field by name. Select **c_nationkey**.
3. Click the field name you selected to view additional configuration options.
   - As the **Scale Type**, select **Categorical**.
   - For the **Transform** selection, choose **None**.

Setup the Y-axis

1. Click ＋ next to the **Y-axis**, then select **c_acctbal**.
2. Click the field name you selected to view additional configuration options.
   - As the **Scale Type**, select **Continuous**.
   - For the **Transform** selection, choose **SUM**.

The visualization is automatically updated as you configure it. The data shown includes only records where the `segment` is `BUILDING`.



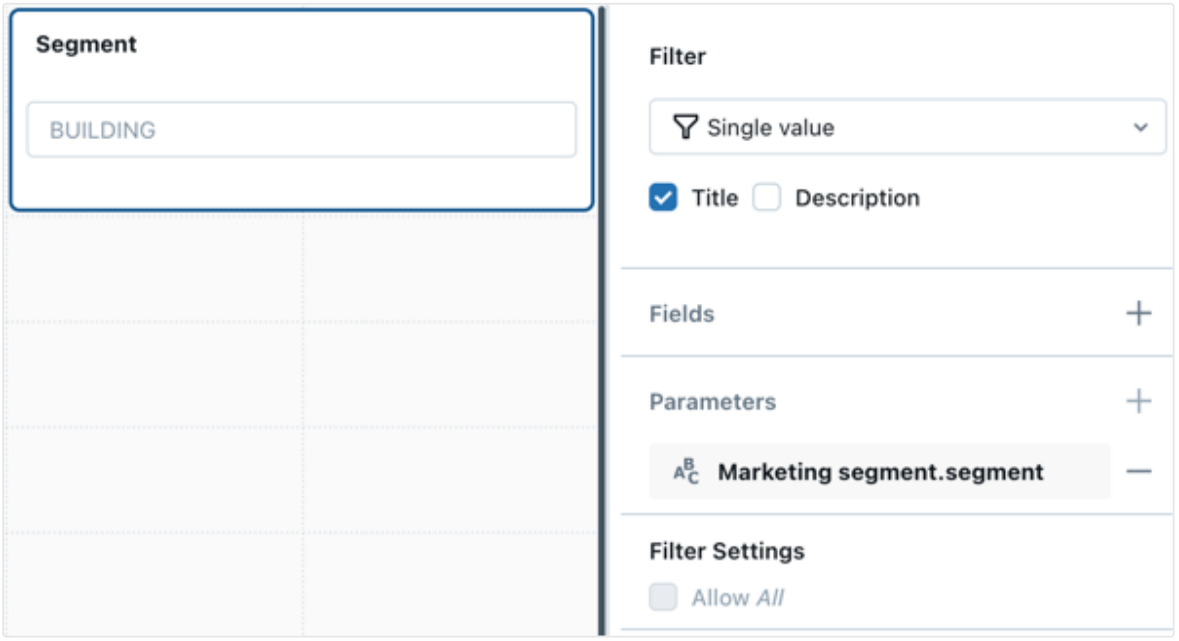
Add a filter

Set up a filter so that dashboard viewers can control which marketing segment to focus on.

1. Click **Add a filter (field/parameter)** to add a filter widget. Place it on the canvas.
2. From the **Filter** drop-down menu in the configuration panel, select **Single value**.
3. Select the **Title** checkbox to show a title field on your filter widget.
4. Click the placeholder title on the widget and type **Segment** to retitle your filter.
5. Click ＋ next to **Parameters** in the configuration panel.

6. Choose **segment** from the **Marketing segment** dataset.

Your configured filter widget shows the default parameter value for the dataset.



Define a selection of values

The filter you created is functional, but it requires the viewer to know the available range of choices before they can type a selection. It also requires that users match the case and spelling when entering the desired parameter value.

To create a drop-down list so that the viewer can select a parameter from a list of available options, create a new dataset to define the list of possible values.

1. Click the **Data** tab.

2. Click **Create from SQL** to create a new dataset.

3. Copy and paste the following into the editor:

SQL

```
SELECT
  DISTINCT c_mktsegment
FROM
  samples.tpch.customer
```

Ask Genie

4. Run your query and inspect the results. The five marketing segments from the table appear in the results.

5. Double-click the automatically generated title, then rename this dataset **Segment choice**.

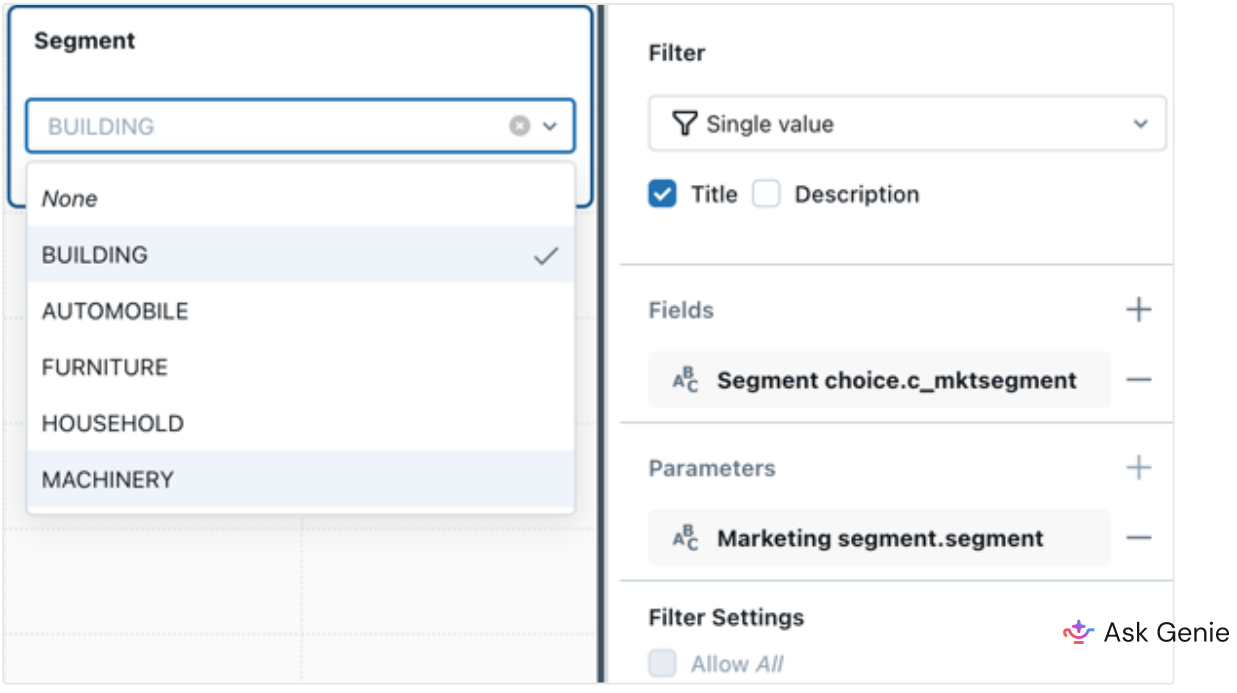
Update the filter

Update your existing filter to use the dataset you just created to populate a drop-down list of values users can select from.

1. Click **Canvas**. Then, click the filter widget you created in a previous step.
2. Click ＋ next to **Fields**.
3. Click **Segment choice**, then click the field name `c_mktsegment`.

Your filter widget updates as you change the configuration. Click the field in the filter widget to see the available choices in the drop-down menu.

> **NOTE**
>
> This tutorial contains a simplified use case meant to demonstrate how to use query-based parameters. An alternate approach to creating this dashboard is to apply a filter to the `c_mktsegment` field.



Next steps

Keep learning about how to work with dashboards with the following articles:

- Learn more about applying filters. See Use dashboard filters.
- Learn more about dashboard parameters. See Work with dashboard parameters.
- Publish and share your dashboard. See Publish a dashboard.

*Last updated on **Apr 9, 2026***

Use dashboard APIs to create and manage dashboards

The Databricks REST API includes management tools specifically for managing AI/BI dashboards. This page demonstrates how to use API tools to create and manage dashboards. To do these tasks using the UI, see Author dashboards.

> **NOTE**
> AI/BI dashboards were previously known as Lakeview dashboards. The Lakeview API still retains that name.

Prerequisites

- Set up authentication to access Databricks resources. To learn about authentication options and get setup instructions, see Authorize access to Databricks resources.
- You need the workspace URL(s) that you want to access. See Workspace instance names, URLs, and IDs.
- Familiarity with the Databricks REST API reference.

Get a draft dashboard

You can use the `dashboard_id` to pull dashboard details from a draft dashboard. The following sample request and response includes details for the current version of the draft dashboard in the workspace.

The `etag` field tracks the latest version of the dashboard. You can use this to verify the version before making additional updates.

```
GET /api/2.0/lakeview/dashboards/04aab30f99ea444490c10c85852f216c

Response:

{
  "dashboard_id": "04aab30f99ea444490c10c85852f216c",
  "display_name": "Monthly Traffic Report",
  "path": "/path/to/dir/Monthly Traffic Report.lvdash.json",
  "create_time": "2019-08-24T14:15:22Z",
  "update_time": "2019-08-24T14:15:22Z",
  "warehouse_id": "47bb1c472649e711",
  "etag": "80611980",
  "serialized_dashboard": "{\"pages\":
[{\"name\":\"b532570b\",\"displayName\":\"New Page\"}]}",
  "lifecycle_state": "ACTIVE",
  "parent_path": "/path/to/dir"
}
```

Update a dashboard

You can use the `dashboard_id` in the previous response to update the new AI/BI dashboard created with that operation. The following example shows a sample request and response. The `dashboard_id` from the previous example is included as a path parameter.

The `display_name` and `warehouse_id` have been changed. The updated dashboard has a new name and assigned default warehouse, as shown in the response. The `etag` field is optional. If the version specified in the `etag` does not match the current version, the update is rejected.

```
PATCH /api/2.0/lakeview/dashboards/04aab30f99ea444490c10c85852f216c

Request body:

{
  "display_name": "Monthly Traffic Report 2",
  "warehouse_id": "c03a4f8a7162bc9f",
  "etag": "80611980"
}
```

```
Response:

{
  "dashboard_id": "04aab30f99ea444490c10c85852f216c",
  "display_name": "Monthly Traffic Report 2",
  "path": "/path/to/dir/Monthly Traffic Report 2.lvdash.json",
  "create_time": "2019-08-24T14:15:22Z",
  "update_time": "2019-08-24T14:15:22Z",
  "warehouse_id": "c03a4f8a7162bc9f",
  "etag": "80611981",
  "serialized_dashboard": "{\"pages\":
[{\"name\":\"b532570b\",\"displayName\":\"New Page\"}]}",
  "lifecycle_state": "ACTIVE",
  "parent_path": "/path/to/dir"
}
```

Create a dashboard

You can use the **Create dashboard** endpoint in the Lakeview API to move your dashboard between workspaces. The following example includes a sample request body and response that creates a new dashboard. The `serialized_dashboard` key from the previous example contains all the necessary details to create a duplicate draft dashboard.

The sample includes a new `warehouse_id` value corresponding to a warehouse in the new workspace. See POST /api/2.0/lakeview/dashboards.

```
POST /api/2.0/lakeview/dashboards

Request body:

{
  "display_name": "Monthly Traffic Report 2",
  "warehouse_id": "5e2f98ab3476cfd0",
  "serialized_dashboard": "{\"pages\":
[{\"name\":\"b532570b\",\"displayName\":\"New Page\"}]}",
  "parent_path": "/path/to/dir"
}

Response:

{
  "dashboard_id": "1e23fd84b6ac7894e2b053907dca9b2f",
```

```
  "display_name": "Monthly Traffic Report 2",
  "path": "/path/to/dir/Monthly Traffic Report 2.lvdash.json",
  "create_time": "2019-08-24T14:15:22Z",
  "update_time": "2019-08-24T14:15:22Z",
  "warehouse_id": "5e2f98ab3476cfd0",
  "etag": "14350695",
  "serialized_dashboard": "{\"pages\":
[{\"name\":\"b532570b\",\"displayName\":\"New Page\"}]}",
  "lifecycle_state": "ACTIVE",
  "parent_path": "/path/to/dir"
}
```

The only required property in the request body is a `display_name`. This tool can copy dashboard content or create new, blank dashboards.

Publish a dashboard

You can use the **Publish dashboard** endpoint to publish a dashboard, set credentials for viewers, and override the `warehouse_id` set in the draft dashboard. You must include the dashboard's UUID as a path parameter.

The request body sets the `embed_credentials` property to `false`. By default, `embed_credentials` is set to `true`. Embedding credentials allows account-level users to view dashboard data. See Publish a dashboard. A new `warehouse_id` value is omitted, so the published dashboard uses the same warehouse assigned to the draft dashboard.

```
POST
/api/2.0/lakeview/dashboards/1e23fd84b6ac7894e2b053907dca9b2f/published

Request body:

{
  "embed_credentials": false
}

Response:

{
  "display_name": "Monthly Traffic Report 2",
  "warehouse_id": "5e2f98ab3476cfd0",
  "embed_credentials": false,
```

```
  "revision_create_time": "2019-08-24T14:15:22Z"
}
```

Publish a dashboard with service principal credentials

You can publish a dashboard with service principal credentials embedded by authenticating as a service principal when making the API call. When you publish using a service principal's token, the dashboard is published with that service principal's data and compute permissions, allowing users without direct data access to view the dashboard.

Before publishing, the service principal must have at least CAN MANAGE permissions on the dashboard, `SELECT` privileges on all data sources used in the dashboard, and CAN USE permissions on the warehouse. For details on creating service principals and generating OAuth secrets, see Service principals and Authorize service principal access to Databricks with OAuth.

First, authenticate as the service principal to obtain an access token:

```
POST https://<databricks-instance>/oidc/v1/token

Request body (form-urlencoded):

grant_type=client_credentials&scope=all-apis

Authorization header:

Basic <base64-encoded-client-id:client-secret>

Response:

{
  "access_token": "eyJraWQiOiJkYTA4ZTVjZ...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Then, use the access token to publish the dashboard with the service principal's credentials:

```
POST
/api/2.0/lakeview/dashboards/1e23fd84b6ac7894e2b053907dca9b2f/published

Authorization header:

Bearer <service-principal-access-token>

Request body:

{
  "embed_credentials": true,
  "warehouse_id": "5e2f98ab3476cfd0"
}

Response:

{
  "display_name": "Monthly Traffic Report 2",
  "warehouse_id": "5e2f98ab3476cfd0",
  "embed_credentials": true,
  "revision_create_time": "2019-08-24T14:15:22Z"
}
```

When `embed_credentials` is set to `true`, viewers of the dashboard use the service principal's permissions to access data and compute resources. Users only need permissions to access the dashboard object itself. All dashboard queries run using the service principal's identity, so audit logs show the service principal as the query executor.

Get published dashboard

The response from GET /api/2.0/lakeview/dashboards/{dashboard_id}/published is similar to the response provided in the previous example. The `dashboard_id` is included as a path parameter.

```
GET
/api/2.0/lakeview/dashboards/1e23fd84b6ac7894e2b053907dca9b2f/published

Response:

{
  "display_name": "Monthly Traffic Report 2",
```

```
  "warehouse_id": "5e2f98ab3476cfd0",
  "embed_credentials": false,
  "revision_create_time": "2019-08-24T14:15:22Z"
}
```

Unpublish a dashboard

The draft dashboard is retained when you use the Lakeview API to unpublish a dashboard. This request deletes the published version of the dashboard.

The following example uses the `dashboard_id` from the previous example. A successful request yields a `200` status code. There is no response body.

```
DELETE
/api/2.0/lakeview/dashboards/1e23fd84b6ac7894e2b053907dca9b2f/published
```

Trash dashboard

Use DELETE /api/2.0/lakeview/dashboards/{dashboard_id} to send a draft dashboard to the trash. The dashboard can still be recovered.

The following example uses the `dashboard_id` from the previous example. A successful request yields a `200` status code. There is no response body.

```
DELETE /api/2.0/lakeview/dashboards/1e23fd84b6ac7894e2b053907dca9b2f
```

> **NOTE**
>
> To perform a permanent delete, use POST /api.2.0/workspace/delete

Next steps

- To learn more about dashboards, see Dashboards.
- See Databricks REST API reference to learn more about the REST API. Ask Genie

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

Last updated on ***Feb 23, 2026***

Manage dashboard permissions using the Workspace API

This page demonstrates how to manage dashboard permissions using the Workspace API. Each step includes a sample request and response and explanations about how to use the API tools and properties together. To manage permissions using the UI, see Share a dashboard.

Prerequisites

- Set up authentication to access Databricks resources. To learn about authentication options and get setup instructions, see Authorize access to Databricks resources.
- You need the workspace URL(s) that you want to access. See Workspace instance names, URLs, and IDs.
- Familiarity with the Databricks REST API reference.

Path parameters

Each endpoint request in this article requires two path parameters, `workspace_object_type` and `workspace_object_id`.

- **`workspace_object_type`**: For AI/BI dashboards, the object type is `dashboards`.
- **`workspace_object_id`**: This corresponds to the `resource_id` associated with the dashboard. You can use the GET /api/2.0/workspace/list or GET

/api/2.0/workspace/get-status to retrieve that value. It is a 32-character string similar to `01eec14769f616949d7a44244a53ed10`.

See Step 1: Explore a workspace directory for an example of listing workspace objects. See GET /api/2.0/workspace/list for details about the Workspace List API.

Get workspace object permission levels

This section uses the **Get workspace object permission levels** endpoint to get the permission levels that a user can have on a dashboard. See GET /api/workspace/workspace/getpermissionlevels.

In the following example, the request includes sample path parameters described above. The response includes the permissions that can be applied to the dashboard indicated in the request.

```
GET
/api/2.0/permissions/dashboards/01eec14769f616949d7a44244a53ed10/permissionL

Response:
{
        "permission_levels": [
                {
                        "permission_level": "CAN_READ",
                        "description": "Can view the Lakeview dashboard"
                },
                {
                        "permission_level": "CAN_RUN",
                        "description": "Can view, attach/detach, and run the
Lakeview dashboard"
                },
                {
                        "permission_level": "CAN_EDIT",
                        "description": "Can view, attach/detach, run, and ed
the Lakeview dashboard"
                },
                {
                        "permission_level": "CAN_MANAGE",
                        "description": "Can view, attach/detach, run, edit,
change permissions of the Lakeview dashboard"
                }
```

```
        ]
}
```

Get workspace object permission details

The **Get workspace object permissions** endpoint gets the assigned permissions on a specific workspace object. See GET /api/workspace/workspace/getpermissions.

The following example shows a request and response for the dashboard in the previous example. The response includes details about the dashboard and users and groups with permissions on the dashboard. Permissions on this object have been inherited for both items in the `access_control_list` portion of the response. In the first entry, permissions are inherited from a folder in the workspace. The second entry shows permissions inherited by membership in the group, `admins`.

```
GET /api/2.0/permissions/dashboards/01eec14769f616949d7a44244a53ed10

Response:
{
"object_id": "/dashboards/490384175243923",
"object_type": "dashboard",
"access_control_list": [
  {
  "user_name": "first.last@example.com",
  "display_name": "First Last",
  "all_permissions": [
        {
          "permission_level": "CAN_MANAGE",
          "inherited": true,
          "inherited_from_object": [
                        "/directories/2951435987702195"
                ]
        }
]
},
{
"group_name": "admins",
"all_permissions": [
        {
                "permission_level": "CAN_MANAGE",
```

```
                "inherited": true,
                "inherited_from_object": [
                        "/directories/"
                ]
        }
]
}
]
}
```

Set workspace object permissions

You can set permissions on dashboards using the **Set workspace object permissions** endpoint. See PUT /api/workspace/workspace/setpermissions.

The following example gives CAN EDIT permission to all workspace users for the `workspace_object_id` in the PUT request.

```
PUT /api/2.0/permissions/dashboards/01eec14769f616949d7a44244a53ed10

Request body:

{
"access_control_list": [
{
        "group_name": "users",
        "permission_level": "CAN_EDIT"
}
]
}
```

For AI/BI dashboards, you can use the group `All account users` to assign view permission to all users registered to the Databricks account. See Share a published dashboard.

Update workspace object permissions

The **Update workspace object permissions** endpoint performs functions similarly to the **Set workspace object permissions** endpoint. It assigns permissions using a `PATCH` request instead of a `PUT` request.

See PATCH /api/workspace/workspace/updatepermissions.

```
PATCH /api/2.0/permissions/dashboards/01eec14769f616949d7a44244a53ed10

Request body:

{
  "access_control_list": [
    {
      "group_name": "account userS",
      "permission_level": "CAN_VIEW"
    }
  ]
}
```

Last updated on ***Feb 23, 2026***

Manage dashboards with Workspace APIs

This tutorial demonstrates how to manage dashboards using the Lakeview API and Workspace API. Each step includes a sample request and response, and explanations about how to use the API tools and properties together. Each step can be referenced on its own. Following all steps in order guides you through a complete workflow.

> **NOTE**
>
> This workflow calls the Workspace API to retrieve an AI/BI dashboard as a generic workspace object. AI/BI dashboards were previously known as Lakeview dashboards. The Lakeview API retains that name.

Prerequisites

- Set up authentication to access Databricks resources. To learn about authentication options and get setup instructions, see Authorize access to Databricks resources.
- You need the workspace URL(s) that you want to access. See Workspace instance names, URLs, and IDs.
- Familiarity with the Databricks REST API reference.

Step 1: Explore a workspace directory

The Workspace List API GET /api/2.0/workspace/list allows you to explore the directory structure of your workspace. For example, you can retrieve a list of all files and directories in your current workspace.

In the following example, the `path` property in the request points to a folder named `examples_folder` stored in a user's home folder. The username is provided in the path, `first.last@example.com`.

The response shows that the folder contains a text file, a directory, and an AI/BI dashboard.

```
GET /api/2.0/workspace/list

Query Parameters:
{
"path": "/Users/first.last@example.com/examples_folder"
}

Response:
{
  "objects": [
    {
      "object_type": "FILE",
      "path": "/Users/first.last@example.com/examples_folder/myfile.txt",
      "created_at": 1706822278103,
      "modified_at": 1706822278103,
      "object_id": 3976707922053539,
      "resource_id": "3976707922053539"
  },
  {
      "object_type": "DIRECTORY",
      "path":
"/Users/first.last@example.com/examples_folder/another_folder",
      "object_id": 2514959868792596,
      "resource_id": "2514959868792596"
  },
  {
      "object_type": "DASHBOARD",
      "path":
"/Users/first.last@example.com/examples_folder/mydashboard.lvdash.json",
      "object_id": 7944020886653361,
      "resource_id": "01eec14769f616949d7a44244a53ed10"
    }
  ]
}
```

Step 2: Export a dashboard

The Workspace Export API GET /api/2.0/workspace/export allows you to export the contents of a dashboard as a file. AI/BI dashboard files reflect the draft version of a dashboard. The response in the following examples shows the contents of a minimal dashboard definition. To explore and understand more serialization details, try exporting some of your own dashboards.

Download the exported file

The following example shows how to download a dashboard file using the API.

The `"path"` property in this example ends with the file type extension `lvdash.json`, an AI/BI dashboard. The filename, as it appears in the workspace, precedes that extension. In this case, it's `mydashboard`.

Additionally, the `"direct_download"` property for this request is set to `true` so the response is the exported file itself, and the `"format"` property is set to `"AUTO"`.

> **NOTE**
>
> The `"displayName"` property, shown in the pages property of the response, does not reflect the visible name of the dashboard in the workspace.

```
GET /api/2.0/workspace/export

Query parameters:
{
  "path":
"/Users/first.last@example.com/examples_folder/mydashboard.lvdash.json",
  "direct_download": true,
  "format": "AUTO"
}

Response:
{
  "pages": [
    {
      "name": "880de22a",
      "displayName": "New Page"
    }
  ]
}
```

Encode the exported file

The following code shows an example response where `"direct_download"` property is set to false. The response contains content as a base64 encoded string.

```
GET /api/2.0/workspace/export

Query parameters:
{
    "path":
"/Users/first.last@example.com/examples_folder/mydashboard.lvdash.json",
    "direct_download": false
}

Response:
{
    "content":
"IORd/DYYsCNElspwM9XBZS/i5Z9dYgW5SkLpKJs48dR5p5KkIW8OmEHU8lx6CZotiCDS9hkppQG
    "file_type": "lvdash.json"
}
```

Step 3: Import a dashboard

You can use the Workspace Import API POST /api/2.0/workspace/import to import draft dashboards into a workspace. For example, after you've exported an encoded file, as in the previous example, you can import that dashboard to a new workspace.

For an import to be recognized as a AI/BI dashboard, two parameters must be set:

- `"format"`: “AUTO” - this setting will allow the system to detect the asset type automatically.
- `"path"`: must include a file path that ends with “.lvdash.json”.

> **IMPORTANT**
>
> If these settings are not configured properly, the import might succeed, but the dashboard would be treated like a regular file.

The following example shows a properly configured import request.

```
POST /api/2.0/workspace/import

Request body parameters:
{
        "path":
"/Users/first.last@example.com/examples_folder/myseconddashboard.lvdash.jso
        "content":
"IORd/DYYsCNElspwM9XBZS/i5Z9dYgW5SkLpKJs48dR5p5KkIW8OmEHU8lx6CZotiCDS9hkppQ
        "format": "AUTO"
}

Response:
{}
```

Step 4: Overwrite on import (Optional)

Attempting to reissue the same API request results in the following error:

```
{
        "error_code": "RESOURCE_ALREADY_EXISTS",
        "message": "Path
(/Users/first.last@example.com/examples_folder/myseconddashboard.lvdash.jso
already exists."
}
```

If you want to overwrite the duplicate request instead, set the `"overwrite"` property to `true` as in the following example.

```
POST /api/2.0/workspace/import

Request body parameters:
{
        "path":
/Users/first.last@example.com/examples_folder/myseconddashboard.lvdash.json"
        "content":
"IORd/DYYsCNElspwM9XBZS/i5Z9dYgW5SkLpKJs48dR5p5KkIW8OmEHU8lx6CZotiCDS9hkppQ
        "format": "AUTO",
        "overwrite": true
}
```

```
Response:
{}
```

Step 5: Retrieve metadata

You can retrieve metadata for any workspace object, including an AI/BI dashboard. See GET /api/2.0/workspace/get-status.

The following example shows a `get-status` request for the imported dashboard from the previous example. The response includes details affirming that the file has been successfully imported as a `"DASHBOARD"`. Also, it consists of a `"resource_id"` property that you can use as an identifier with the Lakeview API.

```
GET /api/2.0/workspace/get-status

Query parameters:
{
        "path":
"/Users/first.last@example.com/examples_folder/myseconddashboard.lvdash.jso
}

Response:
{
        "object_type": "DASHBOARD",
        "path":
"/Users/first.last@example.com/examples_folder/myseconddashboard.lvdash.jso
        "object_id": 7616304051637820,
        "resource_id": "9c1fbf4ad3449be67d6cb64c8acc730b"
}
```

Step 6: Publish a dashboard

The previous examples used the Workspace API, enabling work with AI/BI dashboards as generic workspace objects. The following example uses the Lakeview API to perform a publish operation specific to AI/BI dashboards. See POST /api/2.0/lakeview/dashboards/{dashboard_id}/published.

The path to the API endpoint includes the `"resource_id"` property returned in the previous example. In the request parameters, `"embed_credentials"` is set to `true` so

that the publisher's credentials are embedded in the dashboard. The publisher, in this case, is the user who is making the authorized API request. The publisher cannot embed different user's credentials. See Publish a dashboard to learn how the **Share with data permissions** setting works.

The `"warehouse_id"` property sets the warehouse to be used for the published dashboard. If specified, this property overrides the warehouse specified for the draft dashboard, if any.

```
POST
/api/2.0/lakeview/dashboards/9c1fbf4ad3449be67d6cb64c8acc730b/published

Request parameters
{
  "embed_credentials": true,
  "warehouse_id": "1234567890ABCD12"
}

Response:
{}
```

The published dashboard can be accessed from your browser when the command is complete. The following example shows how to construct the link to your published dashboard.

```
https://<deployment-url>/dashboardsv3/<resource_id>/published
```

To construct your unique link:

- Replace `<deployment-url>` with your deployment URL. This link is the address in your browser address bar when you are on your Databricks workspace homepage.
- Replace `<resource_id>` with the value of the `"resource_id"` property that you identified in retrieve metadata.

Step 7: Delete a dashboard

To delete a dashboard, use the Workspace API. See POST /api/2.0/workspace/delete.

IMPORTANT

This is a hard delete. When the command completes, the dashboard is permanently deleted.

In the following example, the request includes the path to the file created in the previous steps.

```
POST /api/2.0/workspace/delete

Query parameters:
{
        "path":
"/Users/first.last@example.com/examples_folder/myseconddashboard.lvdash.jso
}

Response:
{}
```

Next steps

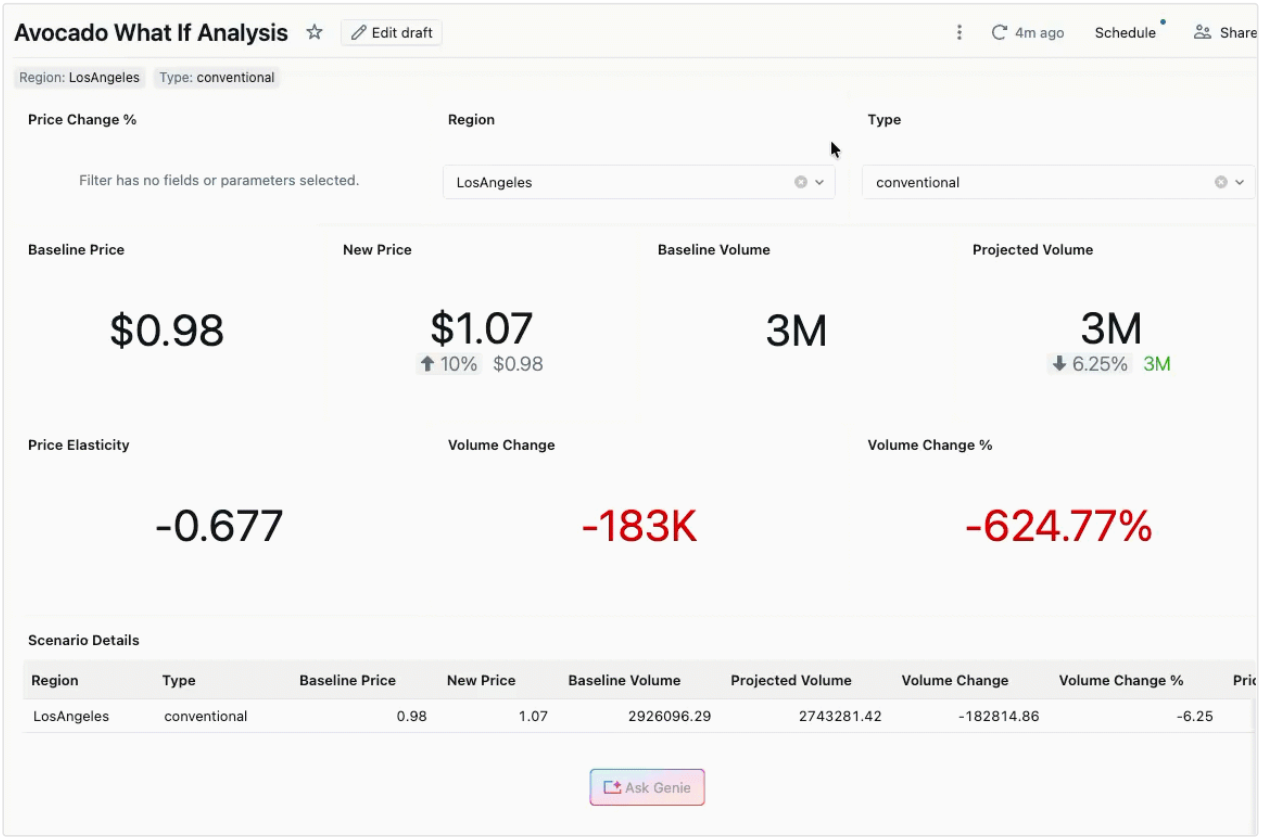
- To learn more about dashboards, see Dashboards.
- To learn more about the REST API, see Databricks REST API reference.

*Last updated on **May 7, 2026***

Tutorial: What If analysis with Genie Code

Analysts are often asked to answer "What If" questions: what happens to volume and total sales if prices increase by 5%? What happens to customer traffic if store hours extend by 30 minutes?

This tutorial shows how to use Genie Code to build an AI/BI dashboard that enables interactive What If analysis. Using avocado sales data, you'll prompt Genie Code to create a dashboard that models how price changes affect weekly volume and total sales by computing price elasticities.



Genie Code responses are not deterministic, so your results might differ along.

Ask Genie

Before you begin

To complete this tutorial, you need:

- Access to an AI/BI dashboard. See Dashboards.
- Genie Code enabled in your workspace. See Use Genie Code for dashboard authoring.
- The following Unity Catalog permissions: `CREATE TABLE` on the target schema, `USE SCHEMA` on the target schema, and `USE CATALOG` on the target catalog.

Understand the approach

To model how prices affect volume and sales, this tutorial uses price elasticity. Elasticity measures the sensitivity of demand to price changes. For example:

- An elasticity of `-1` means that a 1% price increase leads to a 1% decrease in volume.
- A positive price change with low elasticity means demand is relatively stable regardless of price.

You'll prompt Genie Code to compute elasticities from the dataset and build a dashboard where users can input a price change percentage, select a region and avocado type, and instantly see the estimated impact on weekly sales and volume.

Step 1: Upload the avocado dataset to Unity Catalog

This tutorial uses the Hass Avocado Board dataset, which contains weekly avocado sales, prices, and volume split by region across the US.

1. Download the Avocado Prices dataset from Kaggle.
2. Click ⊕ **New > Add or upload data**.
3. Click **Create or modify a table**.
4. Click **browse** or drag and drop the downloaded file onto the drop zone.

5. Select the target catalog and schema in Unity Catalog. You must have `USE CATALOG` on the catalog and `USE SCHEMA` and `CREATE TABLE` on the schema.
6. (Optional) Edit the table name.
7. Click **Create table**.

Step 2: Create a new dashboard

1. Click ⊕ **New** in the sidebar and select **Dashboard**.
2. Enter a name for your dashboard, such as `Avocado What If Analysis`.

Step 3: Open Genie Code

On the dashboard canvas, click the Genie Code icon in the top-right corner to open Genie Code.

Step 4: Submit your initial prompt

PROMPT

Tell Genie Code to do this for you:

```
Help me understand the Avocado dataset. Specifically, I want to model
what would happen if we raised or lowered prices for a particular
region and type. Ideally, I could input a % change in price, a type
of avocado, and a region into this model, and we could see the
corresponding expected change in weekly sales and weekly total volume
by computing the elasticities.
```

Tips for writing effective prompts:

- **Be precise about requirements.** Specify the exact inputs and outputs you want on the dashboard — in this case, inputs for price change percentage, avocado type, and region, and outputs for weekly sales and volume.
- **Describe the context.** Mention the dataset name (for example, "Avocado") so that Genie Code knows which data to search for in Unity Catalog. Ask Genie

- **Ask Genie Code for help.** If you're unfamiliar with a concept, ask Genie Code first. For example: "What are good approaches for modeling how price changes affect volume and total sales?"

Step 5: Review how Genie Code builds the dashboard

After you submit the prompt, Genie Code follows an agentic loop to process your request:

1. **Understands context**: Genie Code reads your prompt and inspects the current dashboard context.
2. **Searches for data**: Genie Code searches for relevant data assets in Unity Catalog. It prioritizes metric views, then falls back to regular tables.
3. **Creates a data model**: For complex calculations like elasticity, Genie Code typically creates a SQL dataset with parameters rather than using custom calculations. It selects the approach best suited to your requirements.
4. **Builds the canvas**: Genie Code edits the dashboard canvas to arrange widgets, visualizations, and input controls.

This loop repeats until Genie Code reaches a complete result. The final dashboard includes parameter inputs for region and price change, and visualizations showing the estimated impact on weekly volume and total sales.

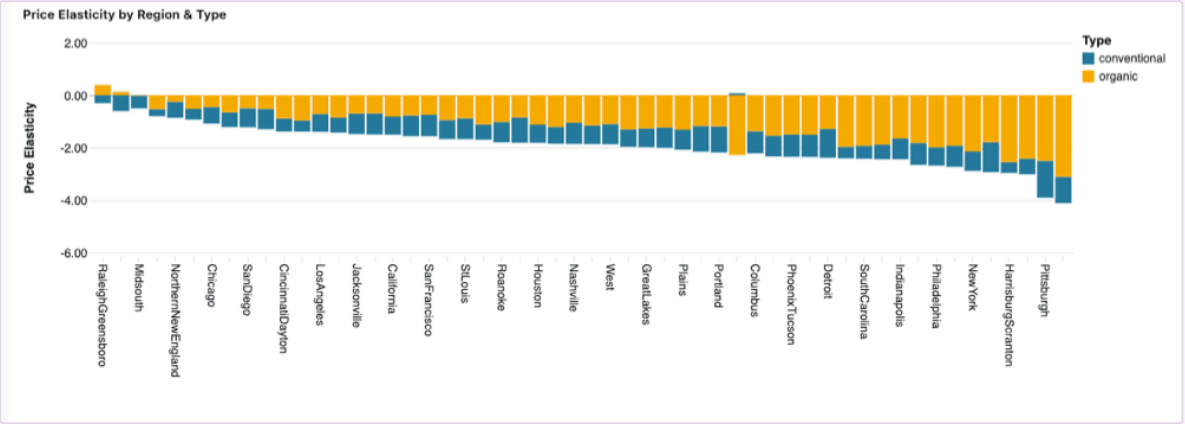
Step 6: Refine the dashboard with follow-up prompts

After reviewing the initial output, use follow-up prompts to improve the dashboard:

- `Add some explanatory text so that users understand what to input and what they're looking at.`
- `Include a representation of model accuracy, such as an R² value.`
- `Add a section comparing elasticities, prices, and volume across different regions.`

Ask Genie

Genie Code can also accept image uploads. If you want to share a screenshot of a particular visualization or external reference, attach it to your prompt for additional context.



Publish and share the dashboard

When you're satisfied with the dashboard, publish it to make it available to others. Published dashboards let users select a region and avocado type, enter a price change percentage, and instantly see the estimated impact on weekly volume and total sales.

To share the dashboard with your team:

1. Click **Publish** in the top-right corner to publish the latest version of the dashboard.
2. Click **Share** to grant access to specific users or groups.
3. (Optional) Set up a schedule to send the dashboard by email. See Manage scheduled dashboard updates and subscriptions.

For more information about publishing and sharing options, see Share a dashboard.

Next steps

- Use Genie Code for dashboard authoring: Learn more about what Genie Code can do for dashboard authoring.
- Create a dashboard: Build a dashboard manually using the UI.
- Use query-based parameters: Set up query-based parameters for interactive filtering.

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

*Last updated on **Apr 21, 2026***

Author dashboards

This page covers the fundamental operations for working with dashboards, including creating, viewing, organizing, and deleting dashboards. You can also create a dashboard using the Dashboard Authoring Agent, see Use Genie Code for dashboard authoring.

For a tutorial on creating a dashboard, see Create a dashboard.

View and organize dashboards

You can access dashboards from the workspace browser along with other Databricks objects.

- Click **Workspace** in the sidebar to view dashboards from the workspace browser. Dashboards are stored in the `/Workspace/Users/<username>` directory by default. Users can organize dashboards into folders in the workspace browser along with other Databricks objects. See Workspace browser.
- To view the dashboard listing page, click **Dashboards** in the sidebar.

  By default, the dashboard listing page shows the dashboards you have access to, sorted in reverse-chronological order. You can filter the list by entering text into the search bar, filtering by last modified within a time period, or filtering by owner.

  You can search across dashboard names, page names, widget titles and descriptions, and dataset names. When using the workspace search bar, you can also search within dataset queries.

- Click a dashboard title to open it. If the dashboard has been published before, the published version opens. Otherwise, the draft dashboard opens.

Create a new dashboard

To create a new dashboard from the dashboard listing page, click **Create** near the upper-right corner of the page.

AI/BI dashboards have the following components:

- **Data**: The **Data** tab allows users to define datasets for use in the dashboard. Datasets are bundled with dashboards when sharing, importing, or exporting them using the UI or API. See Create and manage dashboard datasets
- **Canvas**: The **Canvas** tab can be organized into multi-page reports. Dashboard editors can build and configure their dashboards by adding widgets such as visualizations, filters, text, and images.

Draft and collaborate on a dashboard

New dashboards open as drafts. Changes to a draft dashboard are saved automatically, but they do not automatically sync with the published version if it exists. For details about publishing dashboards, see Publish a dashboard.

To discard edits and restore the draft to the most recently published version, click the ⋮ kebab menu in the upper-right corner of the dashboard, and click **Discard changes**.

You can collaborate on a draft by sharing it with users in your workspace. Users with access interact with the dashboard using their own credentials. You cannot share draft dashboards with users outside the workspace. For more information about permission levels, see Dashboard ACLs.

Create multi-page reports

New dashboards start with a single page named **Untitled page**. To edit the name of a page, double-click the title and enter the new name into the text field. Naming conflicts resolve automatically by appending a number to the title. For information about page limits, see Dashboard limits.

To view the content on a page, click the title to select it.

Add and remove pages

To add a new page:

- Click + plus icon to the right of the current page title on the canvas. By default, your new page is named **Untitled page**.
- (Optional) Double-click the page title and enter a new name to rename the page.

To remove a page:

- Click the ⋮ kebab menu to the right of the page title.
- Click **Delete** to delete the page.

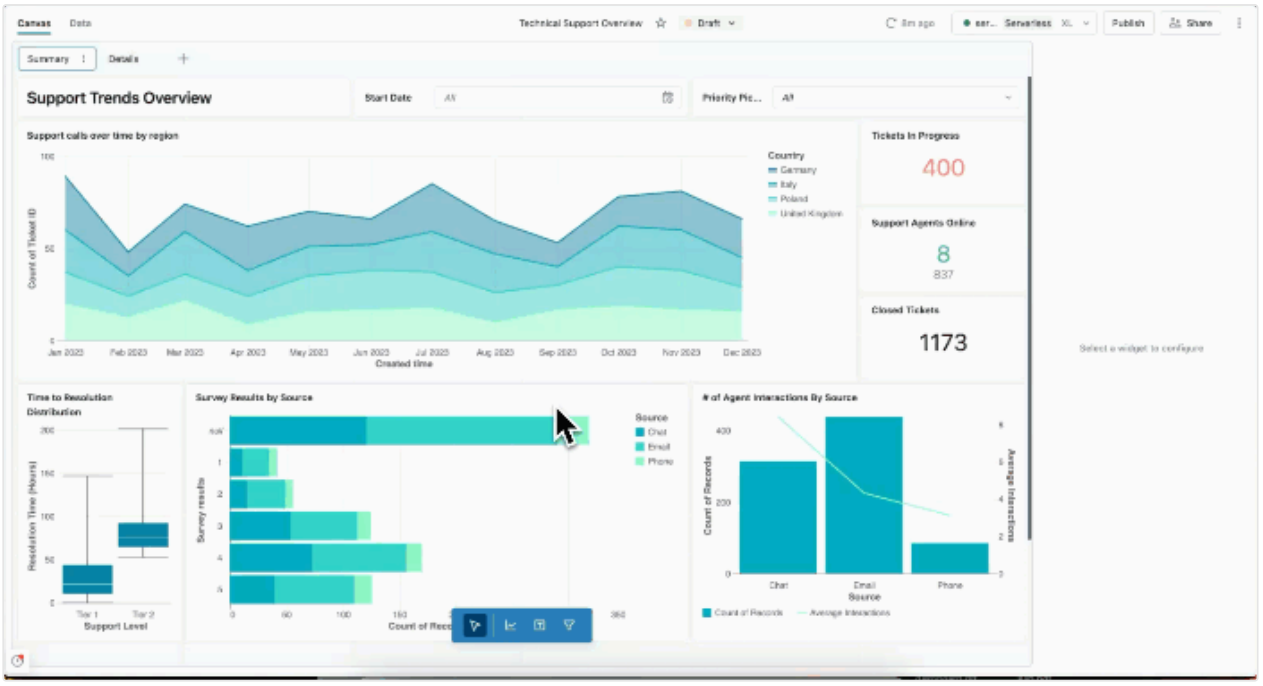
> **NOTE**
> Deleting a page also deletes all of the widgets on that page. If you delete all pages, click **Create a page** to start building your dashboard again.

Clone a page

To clone a page:

1. Click the page title to select it.
2. Click the ⋮ in the title tile, then click **Clone**.

The new page is an exact copy of the original, including all widgets. The underlying datasets remain unchanged.



Copy pages across dashboards

You can copy a page from one dashboard and paste it into another dashboard.

To copy a page across dashboards:

1. In the source dashboard, click the page name in the page tab bar to select it.
2. Press `Command-C` (Mac) or `Ctrl-C` (Windows/Linux) to copy the page.
3. Navigate to the target dashboard and ensure it's in draft mode.
4. Press `Command-V` (Mac) or `Ctrl-V` (Windows/Linux) to paste the page.

The pasted page includes all widgets from the original page. All datasets used by the page are created as new datasets in the target dashboard, ensuring the page functions independently.

Keyboard shortcuts

You can use the following keyboard shortcuts when editing dashboards:

Shortcut	Action
`Command-Z` (Mac) or `Ctrl-Z` (Windows/Linux)	Undo an action on the canvas
`Command-Shift-Z` (Mac) or `Ctrl-Shift-Z` (Windows/Linux)	Redo an action on the canvas
`Command-C` (Mac) or `Ctrl-C` (Windows/Linux)	Copy a selected widget or page
`Command-V` (Mac) or `Ctrl-V` (Windows/Linux)	Paste a widget or page into the current dashboard or a different dashboard
`Delete` key	Delete a selected widget

Copy and remove widgets

You can copy and paste widgets within a dashboard or between dashboards. After you create a new widget, you can edit it as you would any other widget.

To clone a widget on your draft dashboard canvas:

1. Right-click on a widget.
2. Click **Clone**.

A clone of your widget appears below the original.

To remove a widget, select the widget and press the delete key on your keyboard. Or, right-click on the widget and click **Delete**.

Row-level security

AI/BI dashboards enforce data access based on the identity of the viewer. When a dashboard is published with **Individual data permissions**, each viewer's Unity Catalog permissions determine which rows they can see. Row filters and column masks applied

to the underlying tables in Unity Catalog are automatically enforced per user. No additional configuration is required in the dashboard itself.

When a dashboard is published with **Share data permissions**, all viewers see data through the publisher's permissions, so row-level security based on viewer identity does not apply. See What are shared data permissions? for details on publishing options.

Download results

You can download datasets as CSV, TSV, or Excel files. You can download visualizations on the canvas as PNG files.

- To open download options from the **Canvas** tab, click the ⋮ kebab menu in the upper-right corner of the widget.
- To open download options from a **Data** tab, click the ⋮ kebab menu to the right of the dataset.

You can download up to approximately 1GB of results data in CSV and TSV format and up to 100,000 rows to an Excel file. The final file download size might be slightly more or less than 1GB, as the 1GB limit is applied to an earlier step than the final file download.

Delete a dashboard

To delete a dashboard:

1. Open the draft dashboard.
2. Click the ⋮ kebab menu in the upper-right corner of the dashboard.
3. Click **File actions** > **Move to trash**.

Contents of the Trash folder are automatically deleted permanently after *30 days*. To restore an object from the trash, see Restore an object.

You can also organize and delete dashboards from their location in the workspace folder. See Delete an object.

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

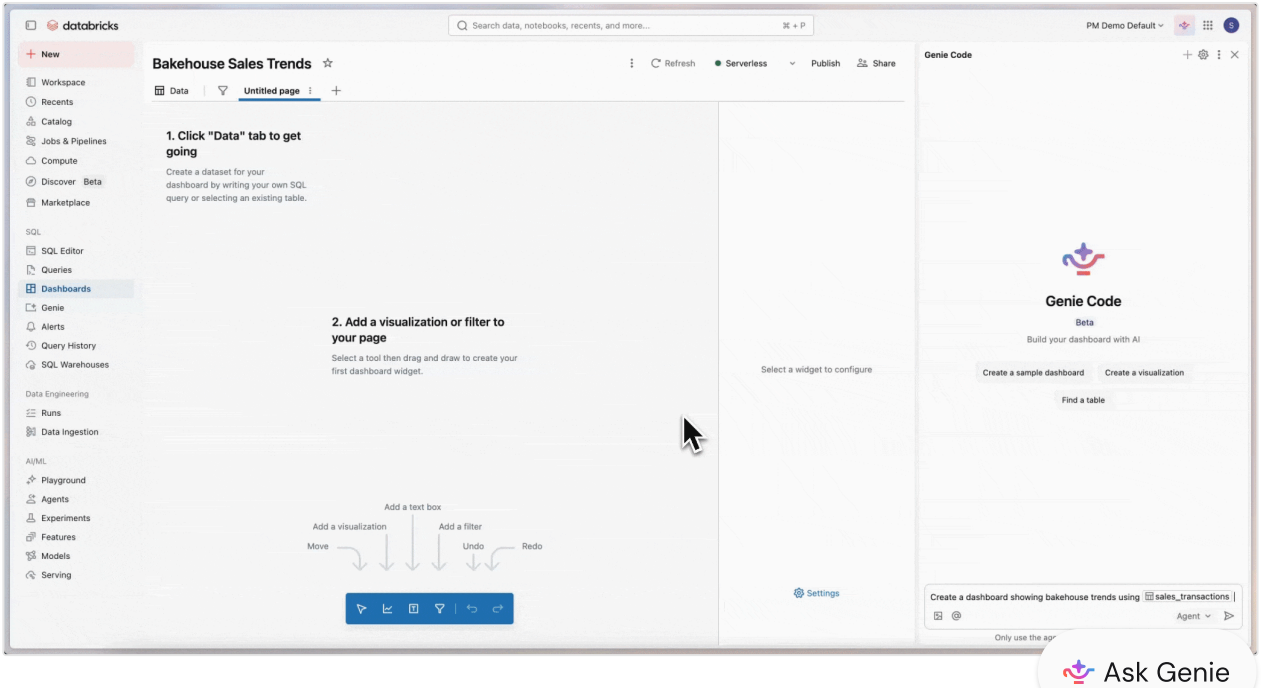
*Last updated on **Apr 15, 2026***

Use Genie Code for dashboard authoring

This page introduces Genie Code for dashboard authoring, an AI data agent available by selecting Agent mode in Genie Code. Designed specifically for AI/BI dashboards, it creates visualizations, configures layouts, and builds complete dashboards—all from a single prompt.

What is Genie Code for dashboard authoring?

Genie Code for dashboard authoring is a powerful capability in Genie Code's Agent mode that transforms Genie Code into an intelligent companion that can automate entire multi-step dashboard authoring workflows.



Compared to Genie Code Chat mode, Agent mode has expanded capabilities including planning solutions, retrieving relevant assets, creating visualizations, configuring multi-page layouts, applying filters, and fixing errors automatically. The agent works with you to approve its plans and confirm next steps before proceeding. With your approval, it can search tables, create datasets, add visualizations, configure filters, and organize dashboard pages.

Genie Code's access and actions are governed by the user's permissions. It can only access data that you have access to and perform operations that you have permissions for.

Requirements

This feature is in Public Preview.

To use Genie Code for dashboard authoring, your workspace needs the following:

- Partner-powered AI features enabled for both the account and workspace. See Partner-powered AI features.
- Your workspace must be in a supported region. Genie Code is a Designated Service that uses Geos to manage data residency. See Geo availability of Genie Code features.
- Genie Code for dashboard authoring preview enabled. See Manage Databricks previews

> **NOTE**
>
> Genie Code for dashboard authoring is not available in workspaces with the compliance security profile enabled. See Compliance security profile.

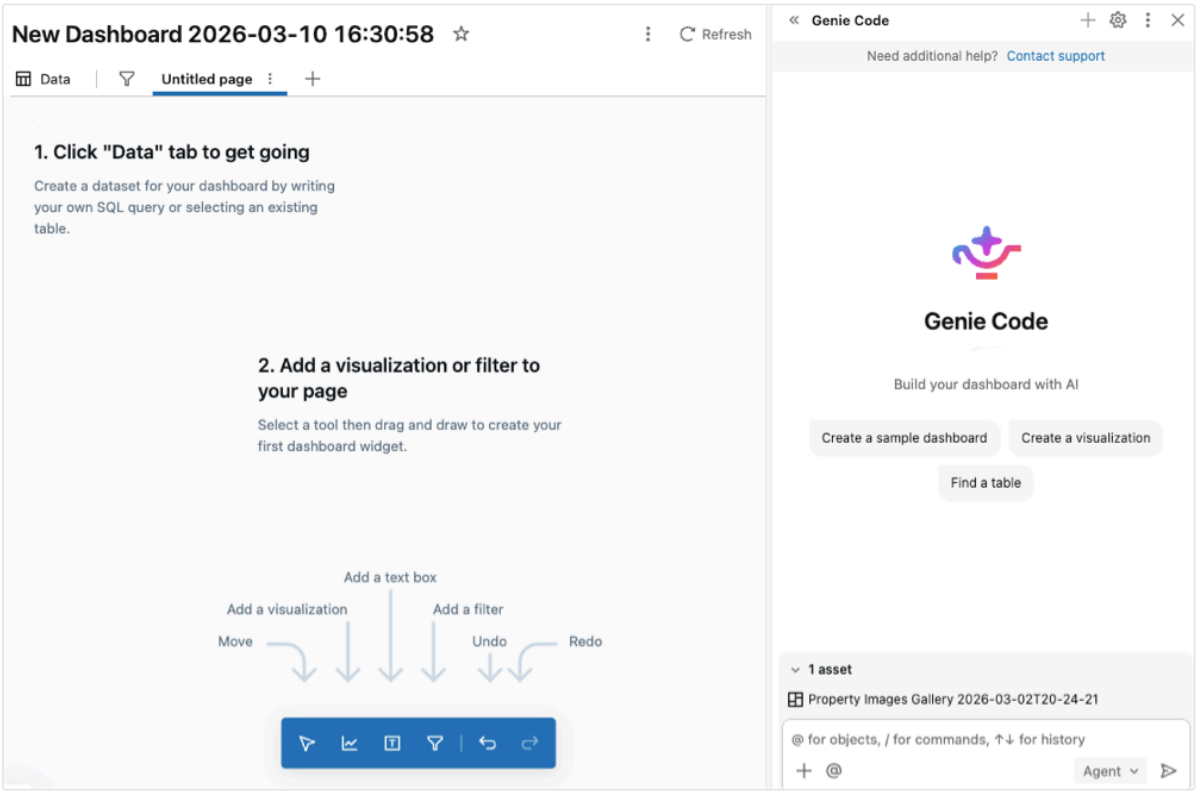
Use Genie Code for dashboard authoring

To use Genie Code for dashboard authoring:

Ask Genie

1. From an AI/BI dashboard, open Genie Code side panel.

2. In the bottom right corner, select **Agent**. This toggles on Genie Code's Agent mode, allowing you to interact with Genie Code for dashboard authoring.



3. Enter a prompt for the agent. For example, "Create a dashboard showing bakehouse trends using `@sales_transactions` from samples.bakehouse."

> **TIP**
>
> Reference specific tables by using `@table_name`. The agent uses that table and any associated metadata to curate its response. The agent respects the user's Unity Catalog permissions, so it can only access the data that you have access to.

4. As the agent generates its response, it often pauses to get your input:

   - For more complex tasks, the agent might create a step-by-step plan and ask clarifying questions. Answer the agent's clarifying questions to help it hone its plan.
   - When the agent needs to create or modify dashboard elements, it asks for your approval before proceeding. **Allow** or **Decline** its request. You can also select **Always allow in current thread** (referring to Genie Code conversation thread) or **Always allow**.

- As the agent continues its work, you might be prompted to select **Continue** or **Reject.** Review the agent's existing work, then select **Continue** to allow the agent to continue to its next steps or **Reject** to tell it to try something else.
- To stop the agent while it is working, click the red ■.

The agent can create new datasets, generate visualizations, configure filters, add pages, and organize dashboard layouts to interpret the results.

> **NOTE**
>
> In order for Genie Code for dashboard authoring to continue its work and take next steps, you need to stay on the current tab the agent is working in.

Use cases

In Agent mode, Genie Code has expanded capabilities, such as finding data, creating visualizations, configuring layouts, and organizing multi-page dashboards.

Genie Code for dashboard authoring can help with complex dashboard authoring tasks, including data exploration, visualization creation, layout design, and filter configuration. You can even create a new dashboard from scratch with Genie Code for dashboard authoring. For better results, provide the agent with context by referencing tables, pipelines, notebooks, queries, and files with `@<resource_name>`. You can also click @ **Add context** to manually select context to provide. Each reference asset persists in the chat context.

Try the following prompts to get started:

- **Dashboard creation**:
  - "Create a dashboard showing bakehouse trends from the `@sales_transactions` table."
  - "Build a sales performance dashboard using `@sales_data` with revenue by region and product category."
  - "Create a customer analytics dashboard from `@customer_data` showing retention, churn, and engagement metrics."
- **Visualization creation**:
  - "Add a line chart showing sales trends over time from this dataset."

    - "Create a bar chart comparing revenue across regions."
    - "Add a pie chart showing product category distribution."
    - "Build a counter widget displaying total revenue for this quarter."
- **Data exploration**:
    - "Which tables contain customer transaction data?"
    - "Show me the first 10 rows of the `@sales_transactions` dataset."
    - "Find a table that contains product inventory data."
- **Dashboard layout and organization**:
    - "Create a new page for regional performance metrics."
    - "Organize these visualizations into a two-column layout."
    - "Add a text widget explaining the key findings from this dashboard."
    - "Clone this page and modify it to show monthly trends instead of weekly."
- **Filters and interactivity**:
    - "Add a date range filter for the entire dashboard."
    - "Create a filter to allow users to select specific product categories."
    - "Add a region filter that applies to all visualizations on this page."
- **Data preparation**:
    - "Create a dataset from `@sales_transactions` that aggregates sales by week and product."
    - "Build a dataset that joins `@customers` and `@orders` tables."
    - "Add a custom calculation to compute the year-over-year growth rate."
- **Dashboard refinement**:
    - "Improve the color scheme to make the charts more readable."
    - "Add titles and descriptions to all visualizations."
    - "Rename this dashboard to 'Q1 2026 Sales Performance'."

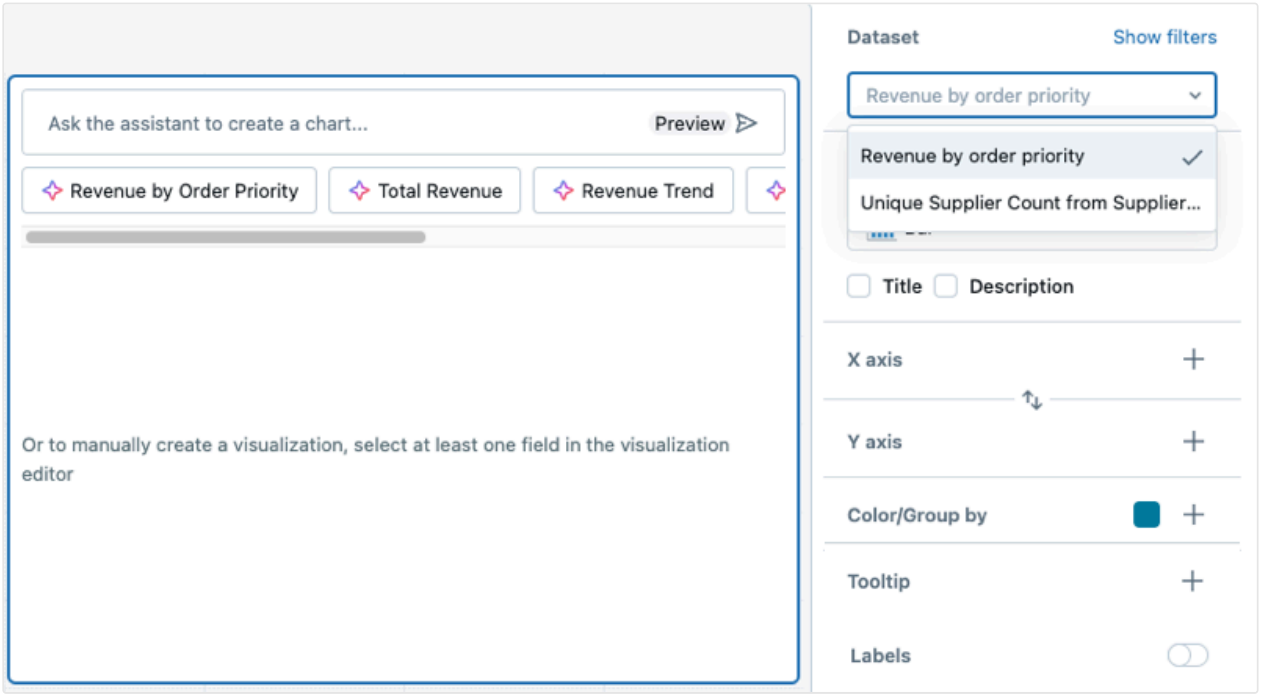
*Last updated on **Apr 21, 2026***

Dashboard visualizations

This page explains how to use and customize visualization widgets on AI/BI dashboards.

Create and configure a chart

After adding a visualization widget to the canvas, the **Visualization Configuration** panel appears on the right side of the screen. By default, the first dataset listed on the **Data** tab is selected, and the default visualization type is a **Bar** chart.



To create a chart, use the following steps:

1. **Select a dataset:** Use the **Dataset** drop-down to choose the dataset for your visualization.
2. **Add filters (optional):** Click **Show filters** to apply a static filter or parameter to your visualization. You can filter on any field. If your dataset includes an option to apply it will also appear.

3. **Choose a visualization type:** Select a visualization type from the **Visualization** drop-down menu. For details about available visualization types and their configurations, see AI/BI dashboard visualization types.
4. **Define x-axis and y-axis fields:** After selecting a dataset, choose the fields to display on the x-axis and y-axis. You can reorder fields by dragging them into the desired position. For chart labels, you can use the display name field to rename the axis label.

> **NOTE**
>
> If you use the Genie Code to generate a chart, the dataset and visualization selections automatically adjust based on your request. For step-by-step guidance on creating AI-assisted visualizations, see Use Genie Code for dashboard authoring.

View the visualization query

While in draft mode, you can view the full SQL query that a visualization widget runs, including any parameters and filters currently applied. This is useful for verifying data accuracy, troubleshooting unexpected results, or understanding how dashboard filters and parameters affect the underlying query.

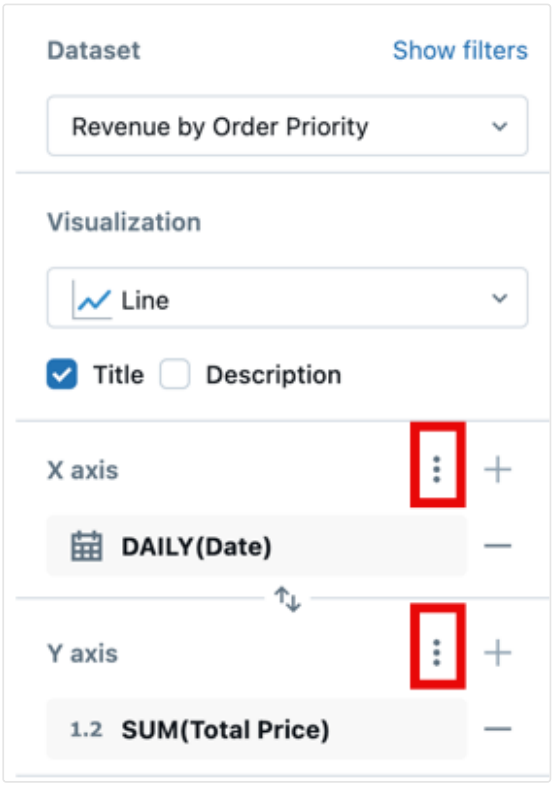
1. Click the ⋮ kebab menu on the visualization widget.
2. Click **View visualization query**.

A modal appears displaying the SQL query for the visualization widget. This query is specific to the widget and only returns the data necessary to render that visualization — it is not the same as the full dataset query. The query reflects the current state of any applied filters. If the underlying dataset uses parameters, parameter values are not substituted into the query. Instead, they appear at the top of the modal for reference.

Apply chart formatting

Charts include customizable formatting options for axes, colors, labels, and tooltips. Formatting options vary by chart type and dataset values.

The following screenshot highlights the kebab menus used for formatting the x and y axes.



Format axis settings

To format the x-axis or y-axis, use the ⋮ kebab menu in the respective **X Axis** or **Y Axis** section of the visualization editing panel.

Configure the following settings:

- **Show axis title:** Enabled by default. Enter a custom title or clear the checkbox to hide the axis title.
- **Show axis values:** Enabled by default. Clear the checkbox to hide axis values.

Depending on the type of data represented, additional controls appear.

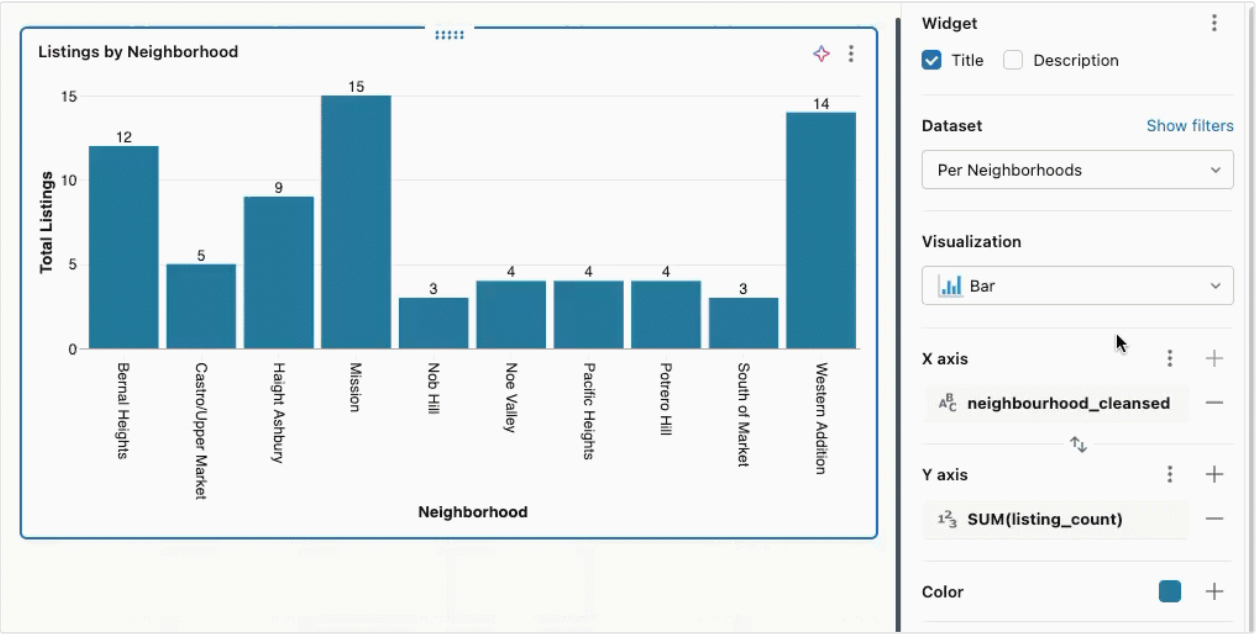
For continuous data:

- **Custom minimum and maximum values:** Type in values to set limits on the displayed data.
- **Reverse axis:** Select **Reverse Axis** to show the axis values in descending order.
- **Scale function:** Choose **Linear** or **Log (Symmetric)**.

- Click **Format** to view additional formatting options. **Auto** automatically formats your chart and is selected by default. Click **Custom** to see additional formatting options. See Format numeric values.

For categorical data:

- **Label Angle:** Choose the angle at which to apply the labels. **Auto** is selected by default.
- **Sort:** Choose how to order categorical values on the axis:
  - **Alphabetical**: Sort category names alphabetically
  - **By field**: Sort by a measure field value in ascending or descending order. After selecting the sort direction, choose which field to use:
    - **Y axis**: Sort by the measure field displayed on the Y axis
    - **Field**: Sort by any other measure field in the dataset, even if it's not displayed on the chart. Select the field from the dropdown and choose the aggregation (SUM, MIN, MAX, or COUNT). This is useful for ordering categories by metrics that provide context without cluttering the visualization.
  - **Custom**: Arrange values in a specific order. Hover over a value and use the grab-handle to drag it into position.



NOTE

Ask Genie

If any of the listed options are unavailable, they cannot be applied to the selected chart type.

Format numeric values

You can format numbers for axis tick labels, data labels, and tooltips. To access formatting options:

1. Click the ⋮ kebab menu next to **X Axis** or **Y Axis** in the chart configuration panel.
2. Click **Format** and choose from the following options:

- **Type:** None, Currency (**$**), Percentage (**%**)
- **Abbreviation:** None, Compact, Scientific
- **Decimal places:** Max, Exact, All, or a custom number of places
- **Group separator:** Optionally, include a comma or other separator.
- **Negative sign:** Choose how negative numbers are displayed. Select **-123.45** (default) for a standard minus sign, or select **(123.45)** to use accounting-style parentheses.

ⓘ NOTE

Different currency formats are available. After selecting **Currency**, choose your preferred option from the drop-down menu.

Customize chart elements

You can format chart legends, colors, line patterns, tooltips, and value labels.

Chart colors

To change chart colors:

1. Click a color in the chart to open the color picker.
2. Enter a HEX or RGB value for an exact color.
3. Click the ⋮ kebab menu in the **Color** section to adjust the opacity of all colors.

Legend

Click the ⋮ kebab menu in the **Color** section to show options for changing the visibility, title, or position of the chart legend. The following options are available:

- **Show legend:** Use the checkbox to show or hide the legend.
- **Show legend title:** Use the checkbox to show or hide the legend title. When the title is visible, you can type to override the default value, which is the column name.
- **Legend position:** Use the drop-down to adjust the legend's placement in the visualization widget. Legends can be positioned at the top, bottom, left, or right of the visualization.

Tooltips

Tooltips display precise measures of the data under the pointer. By default, they show values from the x-axis, y-axis, and any color/grouping fields. To add more fields to a tooltip:

1. Click the the + plus icon in the **Tooltip** section of the visualization editor.
2. Use the drop-down to select additional fields, or use the text entry box to search by name.

> **NOTE**
> If the **Tooltip** section is not present in the visualization editor, that chart type does not support tooltip customization.

Value labels

Use the toggle to turn data labels on or off. When enabled, you can choose how the labels display:

- **Auto**: Displays y-axis values as labels.
- **Field**: Select a specific field to display as labels. You can apply a transformation (such as SUM or AVG) and choose formatting options for the selected field.

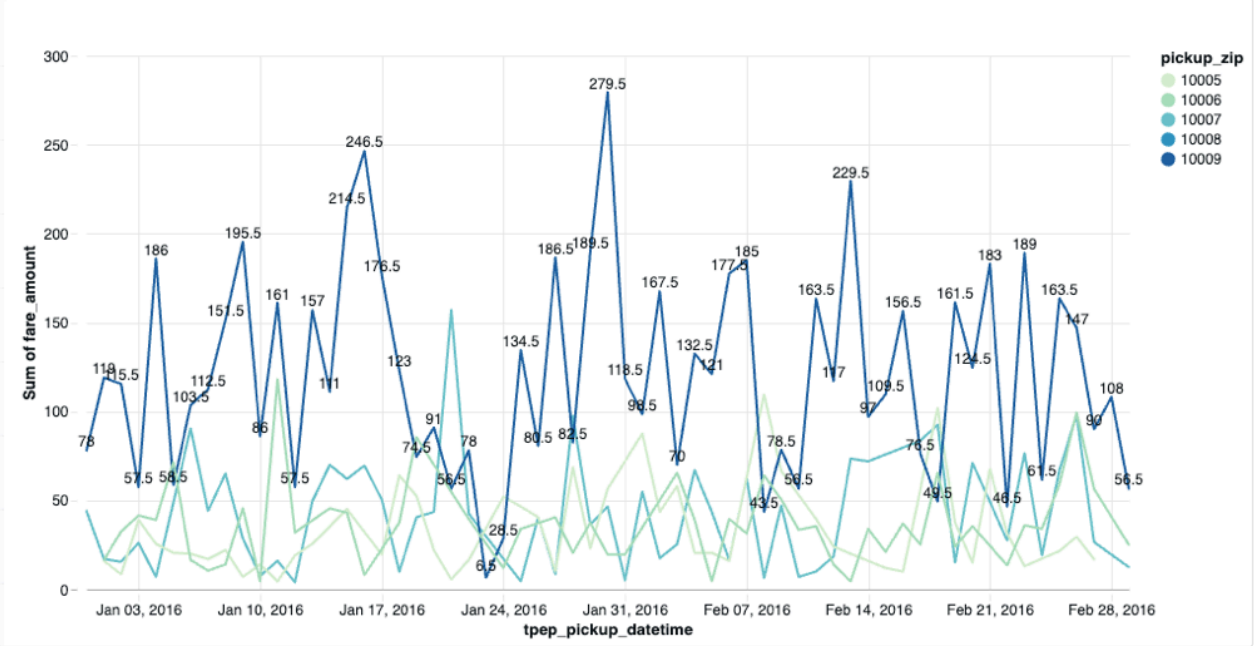
Null values are not displayed as labels on the chart. You can use conditional logic to selectively display labels.

The following query creates a dataset where labels appear only for specific data points:

SQL

```
SELECT
  *,
  CASE
    WHEN pickup_zip = 10009 THEN fare_amount
    ELSE NULL
  END AS custom_label
FROM
  main.samples.taxi_trips
```

When you use `custom_label` as the value label field, only data points where `pickup_zip = 10009` display labels showing the `fare_amount`. All other data points with null values appear on the chart without labels.



Size settings

The **Size** setting controls visual dimensions for line charts, scatter charts, bubble charts, and point maps. Its usage depends on the chart type:

Line charts:

- If no series are specified in the **Size** area, the slider changes all line thickness uniformly.
- If series are specified in the **Size** area, you can set different thickness values for each series.

Scatter and bubble charts:

- Add a field to the **Size** setting to vary the point marker size based on a metric.
- When size is based on a data field, the chart becomes a bubble chart where each point's size reflects the metric value.

Point maps: Use the **Size** setting to vary map marker sizes based on a quantitative field, showing magnitude at different geographical locations.

Gridlines

Charts support toggling gridlines on and off to improve readability. Gridlines appear by default on supported chart types and help viewers interpret data points by providing visual reference lines aligned with axis values.

To toggle gridlines:

1. In the visualization configuration panel, locate the gridline options.
2. Use the toggle to show or hide gridlines for the chart.

Gridlines automatically adjust based on your custom background colors and axis formatting settings.

Annotations

Annotations are horizontal or vertical reference lines that you can add to charts to track thresholds, targets, or benchmarks. For example, you can add a horizontal line at 100 to show your sales target or mark a performance baseline.

The following chart types support annotations:

- Bar

- Line
- Area
- Scatter
- Heatmap
- Box
- Combo
- Histogram
- Waterfall

To add an annotation:

1. In the visualization configuration panel, locate the **Annotation** section.
2. Click the the ＋ plus icon to add a new annotation.
3. Select the line orientation:
   - **Horizontal**: Draws a line parallel to the x-axis at a specified y-axis value
   - **Vertical**: Draws a line parallel to the y-axis at a specified x-axis value
4. Enter the value where the line should appear on the chart.
5. (Optional) Enter a label to describe the annotation. The label appears on the chart next to the line.
6. (Optional) Click the color picker to customize the line color.

You can add multiple annotations to a single chart to track different thresholds or reference points.

Generate a forecast

Use **AI Forecast** (Public Preview) to apply predictive forecasting to line charts to visualize future trends and patterns. Your line chart must have a temporal date field on the x-axis and a single numeric field on the y-axis.

To create a chart with AI forecast:

- With your line chart selected, click the ＋ plus icon to the right of **Forecast**.
- Click **Clone with AI Forecast** in the dialog that appears. A new line chart is created with forecasting applied.

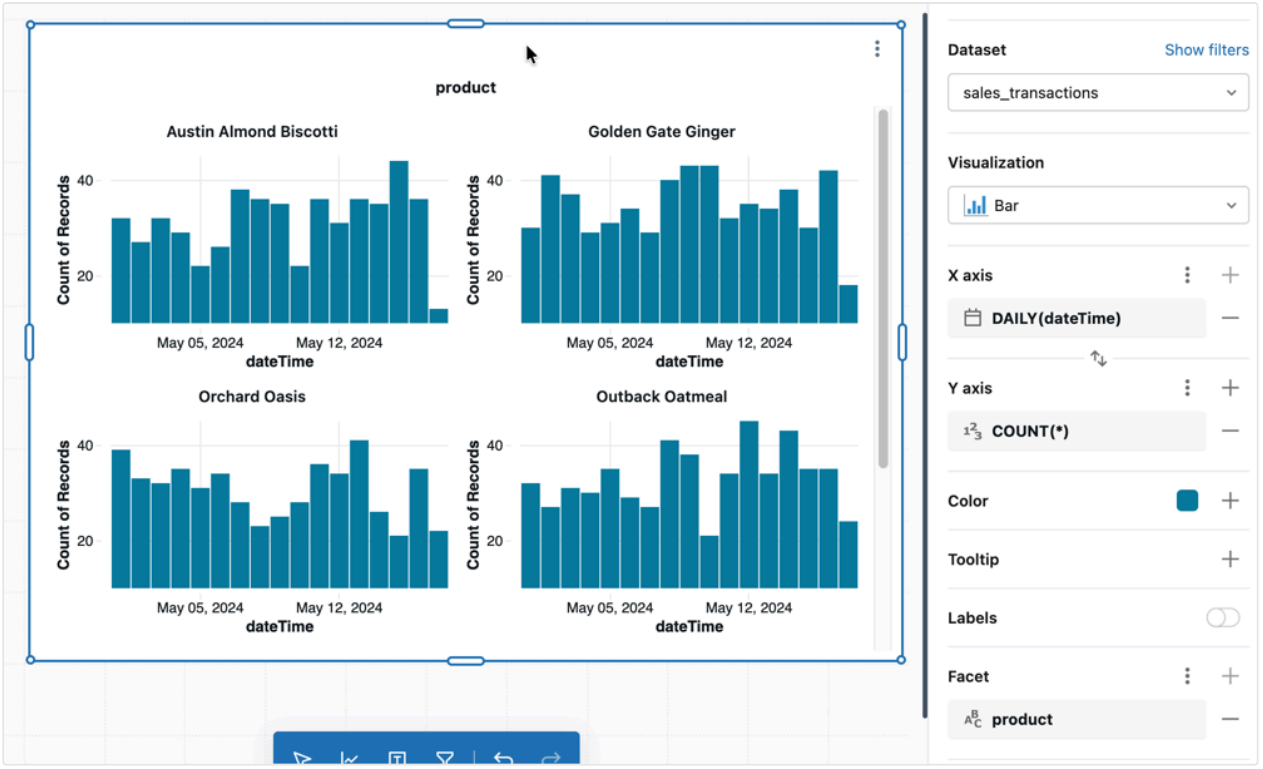
To learn more about the function that generates the forecast, see `ai_forecast` function.

Faceting

Add a dimension to the **Facet** encoding to repeat a chart for all values in that dimension, also known as trellis or small multiple charts. Axes remain synced across all repeated charts, making comparisons easy. To add faceting:

1. Click the the + plus icon in the **Facet** section of the visualization editor.
2. Use the drop-down menu to select a dimension field.

In the ⋮ kebab menu next to **Facet**, you can optionally set the number of rows and columns in the underlying grid. Values that overflow the grid scroll on the chart.



Configuration values: For this example, the following values were set:

- Dataset: `samples.bakehouse.sales_transactions`
- X axis:
  - Field: `dateTime`
  - Transform: `Daily`
- Y axis:
  - Field: `COUNT(*)`
- Facet:
  - Field: `product`

Other visualization types

To learn more about working with table visualizations, see Table and pivot table visualizations.

To learn more about working with map visualizations, see Map visualizations.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **Apr 27, 2026***

Dashboard settings

Dashboard-level settings provide global control over the following aspects of your dashboard:

- **Theme customization**: Control the visual appearance, including colors, fonts, widget styling, and layout preferences
- **General settings**: Add tags (Public Preview), set a locale to control formatting standards, and control the availability of the associated Genie Space.

These settings apply to all widgets and visualizations within a dashboard, ensuring a consistent user experience. When you modify dashboard settings, changes are automatically applied to existing widgets and are used as defaults for any new widgets you add.

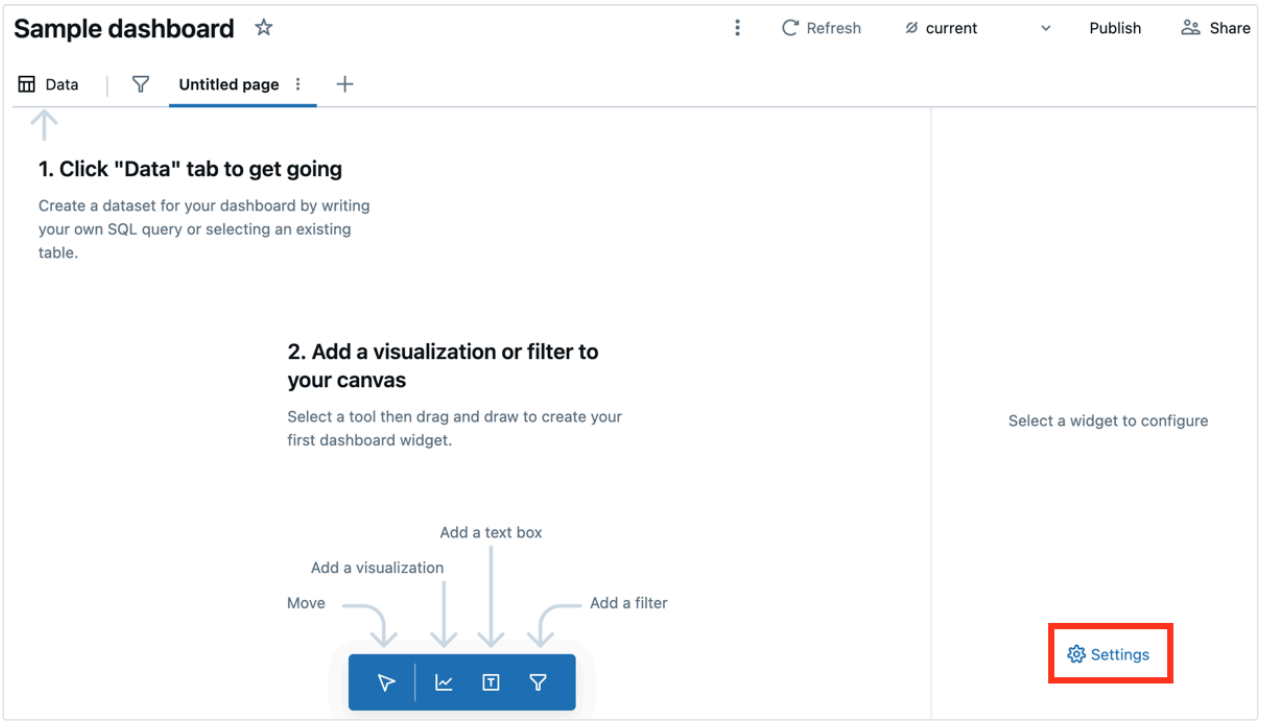
> **NOTE**
> Dashboard settings are only accessible from draft dashboards.

This page explains how to customize dashboard settings and themes to match your organization's branding, improve readability, and ensure consistent data presentation across different regions and languages.

Access dashboard settings

To open dashboard settings:

1. Open your draft dashboard.
2. In the right panel, click **Settings**. This option appears when no widgets are selected.



Theme settings

You can customize the theme by selecting a workspace or preset from the drop-down menu or by adjusting individual settings to create a custom color palette and appearance.

Apply a workspace theme

Workspace themes provide a centralized way to apply consistent colors, fonts, and visual styling across your AI/BI dashboards. All new dashboards automatically inherit the workspace theme. If your workspace admin has configured a workspace theme, you can apply it to your existing dashboard.

For information on workspace themes, see Manage AI/BI dashhoard workspace themes.

Apply the workspace theme to an existing dashboard

To apply the workspace theme:

1. Open your draft dashboard.

2. Access dashboard settings.
3. Use the **Theme** drop-down menu to select **Workspace theme**.
4. Preview the theme in light and dark modes by clicking **Light mode** or **Dark mode** under the **Theme** section.
5. Click **Publish**.

When you apply the workspace theme, your dashboard receives a snapshot of the current theme configuration. The dashboard uses this snapshot even if the workspace admin later updates or deletes the workspace theme.

Choose a preset theme

To choose a dashboard theme:

1. Access dashboard settings.
2. Use the **Theme** drop-down menu to choose a preset theme.

You can preview how your theme will appear in light and dark modes by clicking **Light mode** or **Dark mode** under the **Theme** section.

Customize colors

To adjust individual color settings for the font, canvas background, widget colors, and the visualization palette:

1. Click the color swatch to the right of each setting.
2. Use the color picker or type a HEX or RGB value.

To add or remove colors from the visualization palette, click the + plus icon or — dash icon in the **Visualization palette** section.

Set title alignment

To change the default title alignment for widgets on the dashboard:

1. Click the icon to the right of **Title alignment**.

2. Click **Left**, **Center**, or **Right** to adjust the alignment.

Set a custom font

You can apply a custom font to your dashboard using the **Font Family** setting. The font must already be installed on the dashboard viewer's machine to render correctly. If the font is not available, the dashboard falls back to the Databricks default font.

To set a custom font:

1. Access dashboard settings.
2. Under **Font**, click the **Family** drop-down menu and select **Use local font**.
3. Enter the name of the font exactly as it is installed on the viewer's machine. For example, `Arial`, `Times New Roman`, or `Helvetica Neue`.

General settings

Manage dashboard tags

> **PREVIEW**
>
> This feature is in Public Preview.

Use tags to organize and categorize dashboards for easier management. Tags can be used for automation. For example, you can tag a dashboard as “Work in progress,” and an overnight process can automatically retrieve all dashboards with that tag using the API and assign them to the temporary warehouse until they’re tagged as “Certified.” For more information about tags, see Apply tags to Unity Catalog securable objects.

View tags

To view tags on a dashboard:

1. Click the kebab menu ⋮ near the upper-right corner of the dashboard.
2. Click **Info**.

Add tags

You must have at least CAN EDIT permissions on a dashboard to add a tag. To add a governed tag, you must also have the ASSIGN permission on the governed tag.

To add tags:

1. Open your draft dashboard.
2. In the right panel, click **Settings**.
3. Click **General**.
4. Under **Tags**, add or update a tag:
   - If there are no tags, click the **Add tags** button.
   - If there are tags, click the **Add/Edit tags** icon.
5. Select an existing tag **Key** and **Value** or enter a name of a new tag.
   - Tags that are governed are in the **Governed** section header and have a lock icon.
   - Tag keys are required. Whether a tag value is required depends on the tag key.

Set a locale

You can customize number and date formatting by locale for all visualizations from the canvas of a draft dashboard.

To set a locale:

1. Access dashboard settings.
2. Click **General**.
3. Use the drop-down menu to select your locale.

Configure filter application behavior

You can control when filters are applied to dashboard visualizations. By default, filters apply immediately when a viewer selects a value. You can configure filters to apply

only when the viewer clicks an **Apply** button.

To configure filter application:

1. Access dashboard settings.
2. Click **General**.
3. Under **Apply filters**, select one of the following options:
   - **Instantly**: Filters apply as soon as a viewer selects a value from a dropdown.
   - **With Apply Button**: Filters apply only when the viewer clicks the **Apply** button after making selections.

When **With Apply Button** is selected, viewers can choose multiple filter values before updating the dashboard. This reduces the number of query executions and improves performance for dashboards with multiple filters or large datasets. For more information about filters, see Use dashboard filters.

Enable Genie

You can enable Genie for your dashboard and configure how the Genie Space is created. Genie is enabled by default.

To configure Genie:

1. Access dashboard settings.
2. Click **General**.
3. Toggle **Enable Genie** on.
4. Select one of the following options:
   - **Auto-generate Genie Space** (Preview): Automatically builds a Genie Space using the published dashboard's context. This option is selected by default.
   - **Link existing Genie Space**: Links the dashboard to an existing Genie Space. Paste the URL of the Genie Space you want to link.

*Last updated on **Apr 9, 2026***

Dashboard developer workflows

AI/BI dashboards support programmatic and DevOps-oriented workflows for managing dashboards at scale. You can manage dashboards as code using Declarative Automation Bundles and REST APIs, transfer dashboards across workspaces using import and export, and apply source control using Databricks Git folders.

Capability	Description
Programmatic management	Manage dashboards as code using Declarative Automation Bundles or Terraform. Automate creation, updates, and sharing using REST APIs. Schedule routine dashboard refreshes with Lakeflow Jobs.
Import and export	Export dashboards as portable `.lvdash.json` files and import them into other workspaces. Edit serialized dashboard files directly to make bulk updates without using the UI.
Source control	Version-control dashboard files using Databricks Git folders. Implement CI/CD workflows to develop dashboards in branches and deploy them across environments.

Manage dashboards with Declarative Automation Bundles

To learn how to manage an AI/BI dashboard using Declarative Automation Bundles, see dashboard. For an example bundle that defines a dashboard, see the bundle-examples GitHub repository.

Databricks also offers a Terraform provider. See the Databricks Terraform documentation.

Manage dashboards with REST APIs

See Use Databricks APIs to manage dashboards for tutorials that demonstrate how to use Databricks REST APIs to manage dashboards. The included tutorials explain how to create, manage, and share them.

Schedule updates using Lakeflow Jobs

You can configure a task to routinely update an existing published dashboard. To learn more about orchestrating workflows with Lakeflow Jobs, see Lakeflow Jobs. To learn how to configure a dashboard task, see Dashboard task for jobs.

> **NOTE**
>
> Schedule and subscriber lists that you create using the dashboard UI or API are distinct from scheduling and automation associated with a job. See Automating jobs with schedules and triggers.

Last updated on ***Feb 23, 2026***

Export, import, or replace a dashboard

You can export and import dashboards as files to facilitate the sharing of editable dashboards across different workspaces. To transfer a dashboard to a different workspace, export it as a file and then import it into the new workspace. You can also replace dashboard files in place. That means that when you edit a dashboard file directly, you can upload that file to the original workspace and overwrite the existing file while maintaining the existing sharing settings.

The following sections explain how to export and import dashboards in the UI. You can also use the Databricks API to import and export dashboards programmatically. See POST /api/2.0/workspace/import.

Export a dashboard file

To export a dashboard file:

1. Open a draft dashboard.
2. Click the ⋮ kebab menu at the screen's upper-right corner.
3. Click **File actions** > **Export**.

When the export succeeds, a `.lvdash.json` file is saved to your web browser's default download directory.

Import a dashboard file

- From the dashboards listing page, click ⌄ **> Import dashboard from file**.
- Click **Choose file** to open your local file dialog, then select the `.lvd` you want to import.

- Click **Import dashboard** to confirm and create the dashboard.

The imported dashboard is saved to your user folder. If an imported dashboard with the same name already exists in that location, the conflict is automatically resolved by appending a number in parentheses to create a unique name.

Replace a dashboard from a file

To replace a dashboard from a file:

1. Open a draft dashboard.
2. Click the ⋮ kebab menu at the screen's upper-right corner.
3. Click **File actions** > **Replace**.
4. Click **Choose file** to open the file dialog and select the `.lvdash.json` file to import.
5. Click **Overwrite** to overwrite the existing dashboard.

Edit a dashboard file

The serialized `lvdash.json` file that you have after exporting a dashboard includes complete query syntax and widget settings. In some cases, such as editing an automatically generated page or widget `name` value, it's useful to be able to edit this file directly.

To edit automatically generated page and widget ID values:

1. Export the draft dashboard and open the `.lvdash.json` file in a text editor.
2. Edit the `name` values associated with the page and widget. Save the file.
3. Import the file to your workspace and republish.

> **NOTE**
>
> The `name` value in the JSON file is separate from the `displayName` field, which defines the page name shown in the UI.

*Last updated on **Mar 9, 2026***

Version control dashboards with Git

This page explains how to use Databricks Git folders for version control and collaborative dashboard development. It also describes how to implement CI/CD processes to develop and deploy dashboards across different workspaces.

Overview

This feature is in Public Preview.

Databricks Git folders track dashboard changes and history, support team collaboration, and allow you to deploy dashboards to production and recover previous versions.

Enable dashboard source control

Workspace admins can control workspace access to the Public Preview from the Previews page. By default, the **Support Dashboards in Git folder** preview is **On**.

Alternative: Manual version control

If you cannot enable the Public Preview, use the following manual workflow to track dashboard versions:

1. Export your dashboard as a JSON file. The file format is `lvdash.json`. See Export, import, or replace a dashboard for export instructions.
2. Add that file to a version control system, such as Git.

3. Edit the file. You can edit values in the text file directly or upload it back to your workspace and make changes in the UI.
4. Save the new file. If you've made changes in the UI, export your new file. Use your version control system to track dashboard changes and versions.
5. Update the existing dashboard. From the existing draft dashboard:
   i. Click the ⋮ kebab menu in the upper-right corner and then click **Replace dashboard**.
   ii. Click **Choose file** in the **Replace dashboard from file** dialog. Then, click **Overwrite**.

How Git integration works with dashboards

Databricks Git folders track and manage changes to *draft* dashboards. The dashboard draft reflects all changes in a tracked dashboard. Git doesn't track publishing and scheduling configurations, such as warehouse selection and schedule creation. To manage these configurations, use the UI or automate changes with Declarative Automation Bundles or the AI/BI REST API.

- To use bundles for dashboard management, see dashboard.
- To publish and schedule dashboards with the REST API, see the Lakeview API reference.

> **NOTE**
>
> The Lakeview API uses the previous name for AI/BI dashboards.

Databricks Git folders manage common Git operations for dashboards and other workspace objects. To learn more, see Databricks Git folders.

Applying source control to dashboards

To track dashboards with Git, place them in a Databricks Git folder. Use one of the following options:

- **New dashboards:** Create your dashboard within an existing Databricks Git folder to apply source control from the start.
- **Existing dashboards:** Move an existing dashboard into a Databricks Git folder to track it with Git.

Managing permissions for source-controlled dashboards

Folder-level permissions apply to all objects within that folder, including dashboards. Dashboards in a Git folder inherit the parent folder's permissions in addition to any dashboard-specific permissions. Most Git operations require the CAN MANAGE permission. To learn more, see Folder ACLs and Git folder ACLs.

Recommended development workflow

Clone the repository into your own Databricks Git folder, use feature branches, and submit pull requests. The following table outlines how to use Git folders to manage dashboards during different phases of development and deployment.

> **IMPORTANT**
> Switching Git branches is a destructive operation for dashboards. Databricks removes dashboards that don't exist on the target branch. If you switch back, dashboards reappear with new URLs and IDs, which breaks published links, bookmarks, and API integrations. Verify the target branch before switching and update all references afterward.

Project phase	Workflow	Expected outcome	Known limitations
Initial commit	• Move the dashboard	Git tracks the dashboard in a	

Project phase	Workflow	Expected outcome	Known limitations
	into a Git folder in the workspace. • Commit and push to the remote Git repository.	remote repository.	
Development	• Developers create Git folders on separate dev branches, typically in their home folders. • Commit changes to the development branch. • Merge development branches to main using pull requests.	• Developers work independently. • Git tracks dashboard versions.	Dashboard files use `JSON` format. SQL queries appear as a single line, which can make diffs hard to review in pull requests.
Deployment	• Create a Git folder on the deployment branch in a non-user top-level folder. See CI/CD with	• Consumers access a consistent, published version of the dashboard. • You can share dashboards in	Databricks doesn't provide built-in support for syncing a remote branch with a Git folder in the workspace, or deploying Declarative

Project phase	Workflow	Expected outcome	Known limitations
	Databricks Git folders. • Pull changes into the deployment folder. • Publish dashboards from this folder. • Remove edit+ access and restrict updates to Git. • Share dashboards with consumers.	the same folder with different audiences.	Automation Bundles with a dashboard resource from remote. Set up CI/CD automation to automate: • Pulling updates from the remote repository. • Publishing dashboards after sync. • Deploying Declarative Automation Bundles after an update.

For more best practices on collaboration in Databricks Git folders, see Collaborate using Git folders.

Limitations

Source control with AI/BI dashboards has the following limitations:

- You can commit a maximum of 100 dashboards in a single Git folder. This limit might change during the Public Preview.
- Git-based jobs, such as jobs referencing Git URLs instead of workspace asset IDs or paths, don't work with dashboards.
- Dashboard serialization generates long strings, which makes reading and reviewing differences during pull requests difficult.

- The dashboard file format changes periodically to include new fields and other improvements. During the Public Preview, these changes might appear as differences in Git that you did not initiate.

Last updated on ***Feb 23, 2026***

Monitor dashboard usage

> **PREVIEW**
>
> This feature is in Public Preview.

This page has sample queries that admins can use to monitor activity associated with dashboards. All queries access the audit logs table, which is a system table that stores records for all audit events from workspaces in your region.

Account admins have access to system tables by default. To grant access to other users, see Grant access to system tables.

See System tables reference. For a comprehensive reference of available audit log services and events, see Audit log reference.

Monitor draft and published dashboards

The examples in this section demonstrate how to retrieve audit logs for common questions about dashboard activity.

How many dashboards were created in the past week?

The following query returns the number of dashboards that were created in your workspace over the past week.

SQL

Ask Genie

```
SELECT
```

```
  action_name,
  COUNT(action_name) as num_dashboards
FROM
  system.access.audit
WHERE
  action_name = "createDashboard"
  AND event_date >= current_date() - interval 7 days
GROUP BY
  action_name
```

The following image shows example query results:

	action_name	num_dashboards
1	createDashboard	615

What are the dashboard IDs associated with the most popular dashboards?

Most examples in this article focus on auditing activity on a specific dashboard. You can use audit logs to retrieve specific dashboard IDs. The following query retrieves dashboards with the most views by counting the `getDashboard` and `getPublishedDashboard` actions associated with the IDs.

SQL

```
SELECT
  request_params.dashboard_id as dashboard_id,
  COUNT(*) AS view_count
FROM
  system.access.audit
WHERE
  action_name in ("getDashboard", "getPublishedDashboard")
GROUP BY
  dashboard_id
ORDER BY
  view_count DESC
```

The following image shows example query results:

	dashboard_id	view_count
1	01ef446626ce106080b9328055a6ffa1	21594
2	01ef2a66473f1d2fab5e2741ffa88fb2	21318
3	01ef42e68b281895861df061f399134f	20613
4	01ef540b347b15589f8a379b774e2adf	20491
5	01ef5367ffb1145c89909fed9600c9d9	20477
6	01eeeb9ace481c8f9c1ebfe4940022df	20234
7	01eed19acaae11ab8912d92b63f938b0	20112

How many times was this dashboard viewed in the past week?

The following query uses a specific `dashboard_id` to show the number of times the dashboard was viewed in the past week. The `action_name` column shows whether the draft or published dashboard was accessed. `getPublishedDashboard` refers to views of the published dashboard. `getDashboard` refers to views of the draft dashboard.

For this query, the dashboard ID is provided as a parameter. To learn more about using dashboard parameters, see Work with dashboard parameters. To get the `dashboard_id` for a specific dashboard, see Dashboard URL and ID.

SQL

```
SELECT
  action_name,
  COUNT(action_name) as view_count
FROM
  system.access.audit
WHERE
  request_params.dashboard_id = :dashboard_id
  AND event_date >= current_date() - interval 7 days
  AND action_name in ("getDashboard", "getPublishedDashboard")
GROUP BY action_name
```

The following image shows example query results:

	action_name	view_count
1	getPublishedDashboa...	3220
2	getDashboard	20

What's the number of views by user in the past day?

The following query identifies the number of times a viewer has accessed a dashboard in the past day. The results include whether the user accessed the published dashboard (`getPublishedDashboard`) or the draft dashboard (`getDashboard`).

SQL

```
SELECT
  user_identity.email as username,
  COUNT(user_identity.email) as num_views,
  action_name
FROM
  system.access.audit
WHERE
  service_name = 'dashboards'
AND action_name in ('getDashboard', 'getPublishedDashboard')
AND event_time > now() - interval '1 day'
GROUP BY username, action_name
```

The following image shows example query results:

user_email	action_name	count(action_name)
[redacted]	getPublishedDashboa...	3190
[redacted]@databricks.com	getDashboard	1
[redacted]@databricks.com	getDashboard	3
[redacted]@databricks.com	getDashboard	7
[redacted]@databricks.com	getDashboard	2
[redacted]@databricks.com	getDashboard	6
[redacted]@databricks.com	getDashboard	1

Ask Genie

Who were the top viewers in the past week?

The following query identifies the users who view a specific dashboard most frequently in the past week. It also shows whether those views were on draft or published dashboards. For this query, the dashboard ID is provided as a parameter. To learn more about using dashboard parameters, see Work with dashboard parameters.

SQL

```
SELECT
  user_identity.email as user_email,
  action_name,
  COUNT(action_name) as view_count
FROM
  system.access.audit
WHERE
  request_params.dashboard_id = :dashboard_id
  AND event_date >= current_date() - interval 7 days
  AND action_name in ("getDashboard", "getPublishedDashboard")
GROUP BY action_name, user_email
```

The following image shows example query results:

user_email	action_name	count(action_name)
[redacted]	getPublishedDashboa...	3190
[redacted]@databricks.com	getDashboard	1
[redacted]@databricks.com	getDashboard	3
[redacted]@databricks.com	getDashboard	7
[redacted]@databricks.com	getDashboard	2
[redacted]@databricks.com	getDashboard	6
[redacted]@databricks.com	getDashboard	1

Monitor embedded dashboards

You can monitor activity on embedded dashboards using the audit logs for workspace events. To learn about other workspace events that appear in the audit log, see Workspace events.

The following query retrieves details for dashboards that have been embedded in external websites or applications.

SQL

```
SELECT
  request_params.settingTypeName,
  source_ip_address,
  user_identity.email,
  action_name,
  request_params
FROM
  system.access.audit
WHERE
  request_params.settingTypeName ilike "aibi%"
```

The following image shows example query results:

	settingTypeName	source_ip_address	action_name	request_params
1	aibi_dash_embed_ws_apprvd_domains	172.16.0.255	setSetting	object settingName: "" settingTypeName: "aibi_dash_embed_ws_apprvd_domains" settingKeyName: "6051921418418893" settingKeyTypeName: "workspace"
2	aibi_dash_embed_ws_apprvd_domains	172.16.0.255	setSetting	object settingName: "" settingTypeName: "aibi_dash_embed_ws_apprvd_domains" settingKeyName: "6051921418418893" settingKeyTypeName: "workspace"
3	aibi_dash_embed_ws_apprvd_domains	172.16.0.255	setSetting	object settingName: "" settingTypeName: "aibi_dash_embed_ws_apprvd_domains" settingKeyName: "6051921418418893" settingKeyTypeName: "workspace"
4	aibi_dash_embed_ws_apprvd_domains	172.16.0.255	setSetting	object settingName: "" settingTypeName: "aibi_dash_embed_ws_apprvd_domains" settingKeyName: "6051921418418893" settingKeyTypeName: "workspace"

Set up alerts

You can set alerts to automate this type of monitoring. See Create an alert to learn how to set an alert on a specific threshold.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **Jan 21, 2026***

Dataset optimization and caching

AI/BI dashboards are valuable data analysis and decision-making tools, and efficient load times can significantly improve the user experience. This article explains how caching and dataset optimizations make dashboards more performant and efficient.

Query performance

You can inspect queries and their performance in the workspace query history. The query history shows SQL queries performed using SQL warehouses. Click **Query History** in the sidebar to view the query history. See Query history.

For dashboard datasets, Databricks applies performance optimizations depending on the result size of the dataset. For information about dataset performance thresholds, see Dataset performance thresholds.

Dataset optimizations

Your dashboards optimize for speed by performing filtering and aggregation operations, driven by filters or visualization settings, directly in your browser when possible. These performance optimizations have the following limits:

Dataset Size	Processing Behavior
Small (≤ 100K rows and ≤ 100MB)	For optimal dashboard speed, filtering and aggregation run in your browser after the initial dataset loads. Because these operations are processed locally, they avoid further interaction with the data warehouse and don't appear in the qu

Dataset Size	Processing Behavior
Large (> 100K rows or > 100MB)	Filtering and aggregation are handled on the backend server instead of in your browser. The initial dataset query is wrapped in a SQL `WITH` clause, and the resulting query appears in the query history.
Combined queries (large datasets)	For visualization queries sent to the backend, separate visualization queries against the same dataset that share the same `GROUP BY` clauses and filter predicates are combined into a single query for processing. In this case, users may see a combined query in the query history that fetches results for multiple visualizations or filters.

> **NOTE**
>
> Parameters substitute values directly into a query at runtime, so these operations always appear in the query history.

Caching and data freshness

Dashboards maintain a 24-hour result cache to optimize initial loading times, operating on a best-effort basis. This means that while the system always attempts to use historical query results linked to dashboard credentials to enhance performance, there are some cases where cached results cannot be created or maintained. Cached data has no specific memory limit or fixed query count.

To improve loading times, dashboards first check the dashboard cache. If no cache results are available, they check the generic query result cache. While the dashboard cache can return stale results for up to 24 hours, the query result cache never returns stale data. When the underlying data changes, all query result cache entries are invalidated.

For multi-page dashboards, the following apply:

- Editing a draft dashboard loads and caches all datasets.

- When viewers open a published dashboard, only datasets that support the active page are run and cached.
- If a schedule is set, all datasets refresh according to the schedule, and those results are cached.

The following table explains how caching varies by dashboard status and credentials:

Dashboard type	Caching type
Dashboard published with shared data permissions	Shared cache. All viewers see the same results.
Draft dashboard or dashboard published with individual data permissions	Per user cache. Viewers see results based on their data permissions.

Dashboards automatically use cached query results if the underlying data remains unchanged after the last query or if the results were retrieved less than 24 hours ago. If stale results exist and parameters are applied to the dashboard, queries will rerun unless the same parameters were used in the past 24 hours. Similarly, applying filters to datasets exceeding 100,000 rows prompts queries to rerun unless the same filters were previously applied in the last 24 hours.

Current timestamp functions and cache invalidation

Using `current_timestamp()` or similar functions in your SQL query does not invalidate the dashboard-level cache. However, these functions do invalidate the query result cache, which inspects the SQL query, and trigger a cache refresh.

Scheduled queries

Adding a schedule to a dashboard published with shared data permissions can significantly speed up the initial loading process for all dashboard viewers.

For each scheduled dashboard update, the following occurs:

- All SQL logic that defines datasets runs on the designated time interval.
- Results populate the query result cache and help to improve initial dashboard load time.

*Last updated on **Apr 15, 2026***

Dashboard limits

This article describes the limits that apply to AI/BI dashboards, including pages, datasets, widgets, visualizations, and subscriptions.

Dashboard structure limits

The following limits apply to the overall structure and components of AI/BI dashboards:

Limit	Maximum value	Description
Pages per dashboard	15 pages	A canvas can contain up to fifteen pages. New dashboards start with a single page.
Datasets per dashboard	100 datasets	Define up to 100 datasets per dashboard. Datasets can be created from SQL queries, Unity Catalog tables or views, or uploaded files.
Widgets per dashboard	100 widgets per page	Dashboards can hold up to 100 widgets per page. Widgets include visualizations, filters, text boxes, and images.

Filter limits

The following limits apply to dropdown and multiselect filters on AI/BI dashboards:

Limit	Maximum value	Description
Dropdown and multiselect filter values	100,000 values	Dropdown and multiselect filters can render up to 100,000 distinct values. If the dataset contains more than 100,000 values, any values beyond that limit won't appear in the list but users can still enter them manually. However, the search doesn't return matches for values that exceed the 100,000-value rendering limit.

Visualization rendering limits

The following limits apply to how much data can be rendered in dashboard visualizations:

Visualization type	Maximum rows	Notes
Most visualizations	10,000 rows	To optimize performance, most charts can only render 10K rows on the canvas. Otherwise, visualizations can be truncated. This applies to area, bar, line, scatter, pie, heatmap, and other chart visualizations.
Table visualizations	100,000 rows	Tables can display up to 100K rows before truncation.

> **NOTE**
>
> These row limits apply to the rendering of visualizations on the canvas. For information about dataset size thresholds and performance optimizations, see Dataset performance thresholds.

Dataset performance thresholds

While there's no hard limit on dataset size, Databricks applies different performance optimizations based on dataset result size:

Dataset size	Optimization behavior
Small datasets (≤ 100,000 rows or ≤ 100MB)	The dataset result is pulled to the client, and visualization-specific filtering and aggregation are performed in the browser. Only the dataset query appears in the query history.
Large datasets (> 100,000 rows or > 100MB)	The dataset query text is wrapped in a SQL `WITH` clause, and visualization-specific filtering and aggregation is performed in a query on the backend. The visualization query appears in the query history.

For optimal dashboard performance, keep your datasets small by using explicit column selection, `WHERE` clauses, and parameters to scope your SQL queries. See Dataset optimization and caching.

Dashboard subscription limits

Dashboard subscription emails enforce a 9MB combined attachment size limit that applies across all attached files. The following table describes individual limits.

File type	Limit	Behavior when limit exceeded
All email attachments combined (PDF, DesktopImage, CSV, TSV, and Excel)	9MB combined	See below for specific behaviors based on which files exceed the limit.
Excel attachments	100,000 rows	Rows beyond 100,000 are not included in the attachment.

Subscription size limit behavior

The following describes the expected behavior when the combined file size exceeds the 9MB limit:

- **If the PDF file is greater than 9MB:** The subscription email doesn't include the PDF attachment or any images. It includes a note that says the dashboard has exceeded the size limit and shows the actual file size of the current dashboard.
- **If the combined file size is greater than 9MB:** Only the PDF is attached to the email. The inline message includes a link to the dashboard but no image.
- **If tabular attachments are dropped due to the size limit:** The email body includes the following message: "Attachments exceeded the 9 MB email limit and were excluded. Open the dashboard to download full results."

For more information about dashboard subscriptions, see Manage scheduled dashboard updates and subscriptions.